PCG Arena: A Platform for Blind A/B Testing of Procedural Content Generators with Direct Preference Optimization KOT(Kinga, to dobry tytuł PL?)

(PCG Arena: platforma do zaślepionego testowania A/B generatorów treści proceduralnych z optymalizacją preferencji)

Antoni Czapski

Praca magisterska

Promotor: dr Jakub Kowalski

Uniwersytet Wrocławski
Wydział Matematyki i Informatyki
Instytut Informatyki

March 2026

Abstract

Evaluating procedural content generators (PCGs) for video games poses significant methodological challenges, as player enjoyment is inherently subjective and difficult to quantify through automated metrics alone. This thesis presents PCG Arena, a web-based platform designed for blind A/B testing of Super Mario Bros level generators through human preference data. KOT(gdzieś - może tutaj - zdanie, że Mario has been an important benchmark for a variety of PCG methods, from 2009 until today, albo cos innego ale we wstępie od razu dającego do zroumienia, że to nie jest losowa gierka wzięta bo tak) Super Mario Bros has been used as a benchmark for PCG level generation research since the first Mario AI Competition in 2009, making it an ideal domain for systematic generator comparison. The platform enables players to play two procedurally generated levels side-by-side in their browser, vote for their preferred level, and optionally tag gameplay characteristics. Votes are aggregated using the Glicko-2 rating system, providing uncertainty-aware rankings with confidence intervals.

Building on the collected data, we conduct an exploratory analysis that characterizes generator preference rankings, static expressive range, gameplay trajectory fingerprints, anonymous-user heterogeneity, and tag semantics. The analysis shows that PCG Arena functions not only as a leaderboard, but also as a diagnostic instrument for understanding why generators win or fail. Finally, we present MarioDPO, a level generator fine-tuned with Direct Preference Optimization that achieves a preference score of 0.832, approaching the hand-crafted original Nintendo baseline.

KOT(todo - przeczytać PL potem) Ewaluacja generatorów treści proceduralnych (PCG) dla gier wideo stanowi istotne wyzwanie metodologiczne, ponieważ przyjemność gracza jest z natury subiektywna i trudna do zmierzenia za pomocą automatycznych metryk. Niniejsza praca przedstawia PCG Arenę — platformę internetową do ślepego testowania A/B generatorów poziomów do gry Super Mario Bros poprzez dane o preferencjach graczy. Platforma umożliwia graczom rozgrywanie dwóch proceduralnie generowanych poziomów obok siebie w przeglądarce, głosowanie na preferowany poziom oraz opcjonalne tagowanie cech rozgrywki. Głosy są agregowane za pomocą systemu ratingowego Glicko-2, który zapewnia rankingi z przedziałami ufności.

Na podstawie zebranych danych przeprowadzono analizę eksploracyjną charakteryzującą ranking preferencji generatorów, statyczny zakres ekspresyjny poziomów, trajektorie rozgrywki, heterogeniczność anonimowych użytkowników oraz semantykę tagów. Wyniki pokazują, że platforma może służyć zarówno jako ranking generatorów, jak i narzędzie diagnostyczne ujawniające ich tryby porażki. Finalnie, praca przedstawia MarioDPO — generator poziomów wytrenowany metodą Direct Preference Optimization, który osiąga wynik preferencji 0.832, zbliżony do oryginalnych poziomów Nintendo.

Contents

Chapter 1

Introduction

1.1 Procedural Content Generation in Video Games

Procedural Content Generation (PCG) refers to the algorithmic creation of game content—levels, maps, characters, items, narratives—with limited or indirect KOT(OK, ale nie zacytowanie https://www.pcgbook.com to zbrodnia w kontekscie tej pracy :P) human input [32, 31][24]. PCG has a long commercial history: *Rogue* (1980) and *Elite* (1984) used algorithmic generation to overcome memory constraints, and modern titles such as *Minecraft*, *No Man's Sky*, and *Spelunky* rely on procedural generation as a core design pillar, providing variety, replayability, and vast explorable worlds. Academic interest in PCG has grown steadily since the mid-2000s, producing a rich landscape of generation techniques spanning constructive rules, evolutionary search, formal grammars, neural networks, and reinforcement learning.

Among the many game domains studied, Super Mario Bros (SMB) has accumulated the largest body of comparative work in PCG level generation research. Its 2D tile-grid format provides a rich yet tractable design space, its physics are well understood, and the open-source Mario AI Framework [10] offers a standardised environment for both automated and human evaluation.KOT(w sumie mało obrazków - może screenik z Mario żeby ludzie widzieli jak nie wiedzą :P) Over 15 years of research has produced dozens of Mario level generators spanning most major PCG paradigm—yet evaluating and comparing them remains an open challenge.

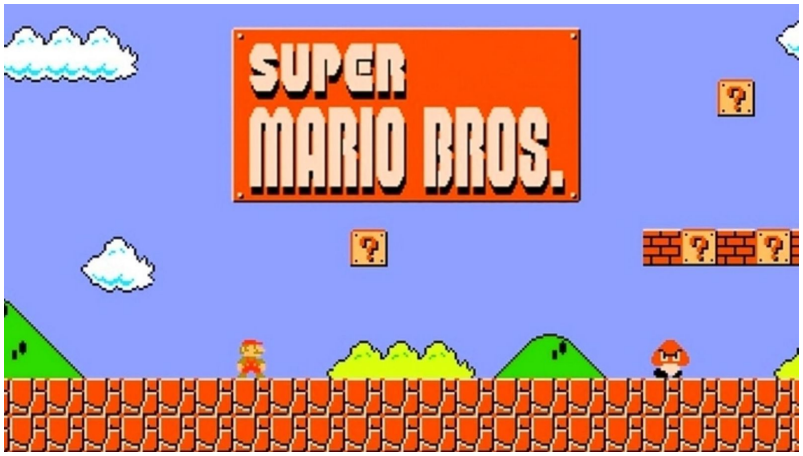


Figure 1.1: Screenshot of the original *Super Mario Bros* (Nintendo, 1985). The player controls Mario through side-scrolling levels featuring platforms, enemies, power-ups, and coins. The 2D tile-grid structure of these levels provides the design space studied throughout this thesis.

1.2 The Evaluation Problem

Evaluating procedural content generators is fundamentally difficult because the ultimate quality criterion—player enjoyment—is subjective, multidimensional, and expensive to measure. The PCG literature has developed three families of evaluation methods, each with significant limitations.

Automated structural metrics, such as linearity, leniency, and density, can be computed directly from level tilemaps and are widely used for characterising generator output [25, 9]. However, Mariño et al. [12] demonstrated that these metrics only weakly correlate with human-rated difficulty and enjoyment, concluding that "current computational metrics should not be used in lieu of user studies." Multiple incompatible definitions of the same metric—at least three formulations of leniency exist in the literature—further complicate cross-study comparisons.

Agent-based testing uses AI controllers—typically the A* agent from the 2009 Mario AI Championship—as player proxies. Agent-derived features such as jump count or completion fraction provide scalable difficulty estimates [33], but superhuman agents can solve levels that human players cannot. Difficulty estimates depend on the specific agent policy, introducing a systematic bias that is rarely quantified.

Human evaluation studies remain the gold standard but are expensive and typically small-scale: controlled lab studies use 15–37 participants [21, 12], while larger web-based studies sacrifice experimental control [16]. Small samples cannot control for varying levels of gaming experience, diverse player expectations, or learning effects across successive trials. Critically, all existing studies were conducted as

one-time events—often years apart—rather than as a persistent evaluation infrastructure, and the largest web-based studies predate the wave of ML-based generators (GANs, LLMs, diffusion models) that have since reshaped the field. Many of these studies also reveal which algorithm produced each level to participants, even though, when players know the source generator, expectations contaminate their judgments. KOT(I suppose how the original Mario PCG competition worked? exactly as your study - play two levels choose best?)

This persistent gap between scalable-but-inaccurate automated metrics and valid-but-expensive human studies motivates the work presented in this thesis.

1.3 Research Questions and Contributions

This thesis addresses the PCG evaluation gap through the design, deployment, and analysis of *PCG Arena*, a web-based platform for blind A/B testing of Mario level generators. We investigate five research questions:

RQ1: Human preference ranking
Which generators are preferred by players in blind pairwise comparisons?

RQ2: Static expressive range
How do generators differ structurally, and which static properties align with player preference?

RQ3: Gameplay footprint
What behavioral signatures appear in player trajectories for each generator?

RQ4: User heterogeneity
Are anonymous player preferences and play styles homogeneous, or do visible subgroups emerge?

RQ5: Tag semantics
What do optional tags reveal about why levels win or lose?

KOT(probably noindent here to make it look better)

The thesis makes four contributions:

1. **PCG Arena**, a persistent, open-access web platform for blind A/B testing of Super Mario Bros level generators, featuring browser-based gameplay, Glicko-2 uncertainty-aware rankings, and a novel matchmaking algorithm (AGIS).
2. **A user study** analysing 1,000 exported pairwise votes from 77 anonymous browser/player IDs across 14 research generators, with 1,968 played side records, embedded gameplay trajectories, and subjective tags.

3. **An exploratory analysis pipeline** connecting blind preference votes to static expressive-range metrics, trajectory fingerprints, anonymous-user preference maps, and qualitative tag-based failure-mode diagnostics.

4. **MarioDPO**, a level generator fine-tuned with Direct Preference Optimisation on arena votes, achieving a preference score of 0.832 and approaching the handcrafted Nintendo baseline.

1.4 Thesis Structure

The remainder of this thesis is organised as follows. Chapter 2 surveys background and related work on procedural content generation `KOT((in the context of Maio level generation?))` in the context of Mario level generation, evaluation methodology, rating systems, and preference learning. Chapter 3 describes the design and implementation of the PCG Arena platform. Chapter 4 presents the user study and an exploratory data analysis addressing the five research questions. Chapter 5 details the development of the Judge Function and the MarioDPO generator. Chapter 6 summarises contributions, discusses limitations, and outlines future directions.

Chapter 2

Background and Related Work

2.1 Super Mario Bros

Super Mario Bros (SMB), released by Nintendo in 1985, is a side-scrolling platform game in which the player controls Mario through a sequence of levels. Each level is composed of a 2D tile grid, typically 16 tiles tall and 150–210 tiles wide, scrolling horizontally from left to right. The game's design vocabulary includes several categories of elements:

- **Terrain and platforms.** Solid ground blocks (`X`), brick blocks (`S`) that can be broken by hitting from below, pyramid blocks (`#`), and jump-through platforms (`%`) form the physical structure of each level.
- **Interactive blocks.** Question blocks may contain coins or power-ups: a Super Mushroom (which transforms Small Mario into Super Mario, allowing him to break bricks and survive one extra hit), a Fire Flower (enabling ranged attacks), or a 1-Up Mushroom (extra life). Invisible blocks provide hidden rewards.
- **Enemies.** Goombas are defeated by jumping on them; Koopa Troopas retract into their shells, which can then be kicked as projectiles; Spikies (Spinies) cannot be stomped; and Bullet Bills are fired from cannons. Winged variants of each enemy add vertical movement.
- **Collectibles.** Coins are scattered across levels as floating items or hidden inside blocks. Collecting 100 coins grants an extra life.
- **Pipes.** Decorative or functional vertical structures, some of which spawn Piranha Plant enemies.
- **Level boundaries.** Each level begins at a Mario spawn point (`M`) and ends at a flag pole (`F`).

The game's physics—gravity, inertia, jump arcs, and collision detection—are deterministic and well-characterised. These properties make SMB an ideal benchmark for PCG research: the tile-grid format provides a discrete, tractable design space; the mix of terrain, enemies, and power-ups yields a combinatorially rich yet well-

understood design vocabulary; and the game's cultural familiarity means that nearly all experimental subjects already understand the controls and goals (Figure 1.1).

2.2 Procedural Content Generation

Procedural Content Generation (PCG) is the algorithmic creation of game content with limited or indirect human input [32]. Drawing on the taxonomies presented by Togelius et al. [32] and Summerville et al. [31], we distinguish five broad generation paradigms: KOT(trche nie wiem jak z tych obu paperów można powiedzieć że jest takich 5 paradygmatów generacji... przejrzałem je teraz tylko żeby sobie przypomnieć ale mi się to tochę nie spina z tekstami źródłowymi)

Constructive / Rule-based
Sequentially place content according to hand-authored rules or grammars. Fast and reliable, but limited in expressiveness.

Search-based (SBPCG)
Frame generation as optimisation over a fitness function, typically using evolutionary algorithms. Flexible but computationally expensive.

Grammar / Pattern-based
Use KOT(of?) formal grammars, design patterns, or n-grams to encode structural idioms. Captures human design style but requires pattern libraries.

Machine Learning (PCGML)
Train neural networks on existing human-designed content. Data-driven and expressive, but may lack diversity or playability guarantees.

Reinforcement Learning (PCGRL)
Frame level design as a Markov Decision Process; train an agent to construct content. Unlike PCGML, requires no corpus of existing levels—only a reward function that encodes the desired content properties.

These paradigms are not mutually exclusive; many modern systems are hybrids. For instance, MarioGAN [33] combines a learned GAN with CMA-ES search in its latent space. A further useful distinction is between *offline* generation (content precomputed before the game ships) and *online* generation (content created at runtime, potentially adapting to the player). Online generation enables personalisation but imposes stricter time budgets—a theme central to experience-driven PCG [34]. A desirable property of any generator is *controllability*: the ability to accept parameters describing desired content features and produce content that complies.KOT(czemu contrallability jest nagle boldem jak wczesniej online / offline było emphem?)

2.2.1 Search-Based Approaches

Search-Based PCG (SBPCG) frames content generation as an optimisation problem [32]. A *fitness function* encodes the desired properties of the content (e.g., playability, target difficulty, aesthetic criteria), and an evolutionary algorithm—typically a genetic algorithm or CMA-ES—searches a space of candidate solutions to maximise it.

The approach is attractive because fitness functions provide explicit, modular designer control: one can compose playability constraints, difficulty targets, and diversity objectives into a single optimisation criterion. However, SBPCG has two well-known limitations. First, each fitness evaluation may require an expensive simulation (e.g., running an A* agent through a candidate level). Second, designing good fitness functions is itself a difficult problem—one that mirrors the evaluation challenge motivating this thesis.

The Mario AI Level Generation Competition [21] featured several search-based entries. More recent work has applied CMA-ES to the latent space of trained generative models [33], effectively combining search-based and ML-based paradigms.

2.2.2 Grammar-Based and Rule-Based Approaches

Constructive generators build content sequentially according to hand-authored rules, typically working left-to-right through a level. The Notch generator shipped with *Infinite Mario Bros* [10] is the canonical example: it iterates through section templates (straight, hill, tubes, gaps, cannons) with fixed selection probabilities and basic playability checks. Constructive methods are fast and guarantee structural validity, but their expressive range is narrow.

Grammar-based approaches extend this idea by encoding design rules in a formal grammar. Shaker et al. [23] used **Grammatical Evolution** (GE) to map integer genotypes to level phenotypes via context-free grammar productions, combining the interpretability of grammars with evolutionary search. Smith and Whitehead [25] introduced **Launchpad**, a rhythm-based generator that maps player actions to level geometry via a design grammar. The same paper introduced **Expressive Range Analysis** (ERA)—the practice of generating a large sample of levels, computing structural metrics, and visualising their joint distribution—which has become the dominant method for characterising PCG generators.

2.2.3 Machine Learning-Based Generation (PCGML)

Machine learning approaches to PCG (PCGML) train neural networks on existing content to learn implicit design knowledge [31]. Several architectures have been applied to Mario level generation:

Sequence models. Summerville and Mateas [29] trained a 3-layer LSTM to generate levels column-by-column using a "snaking" tokenisation that preserves vertical adjacency. Co-generating levels with A*-agent play traces achieved 97% playability.

Generative adversarial networks. MarioGAN [33] trains a Deep Convolutional GAN on sliding-window segments from a single original SMB level. CMA-ES then searches the GAN's latent space to optimise fitness functions based on tile distributions or agent-derived playability. This work established the **Latent Variable Evolution** (LVE) paradigm.

Large language models. MarioGPT [28] fine-tunes DistilGPT-2 on column-tokenised Mario levels with text-prompt conditioning (e.g., "many pipes, many enemies, low elevation"), enabling the first natural-language-controllable level generator. 88% of generated levels are playable without post-processing.

Diffusion models. MarioDiffusion [19] uses a text-conditioned UNet denoiser to generate 16×16-tile scenes from natural-language captions, representing the latest generation of multimodal PCG.

Reinforcement learning. PCGRL [11] formulates level design as an MDP and trains an RL agent to edit tile grids, bypassing the need for training data entirely.

2.3 Super Mario Bros as a PCG Benchmark

Several practical factors explain why Super Mario Bros recurs throughout the PCG literature. Its 2D tile-grid format provides a rich yet tractable design space, its physics introduce no new mechanics after the first level (isolating level design as the sole variable of interest), nearly all experimental subjects are familiar with the game's controls and goals, and the open-source Mario AI Framework provides a standardised evaluation environment.

2.3.1 The Mario AI Framework

The Mario AI Framework [10] is an open-source Java implementation based on *Infinite Mario Bros*, a public-domain clone of Nintendo's Super Mario Bros created by Markus Persson in 2008. It served as the basis for three tracks of the Mario AI Championship (2009–2012):

1. **Gameplay Track**—build a controller that plays levels as well as possible.
2. **Learning Track**—build a controller that learns to play.
3. **Level Generation Track**—build software that generates levels for human players.

The Level Generation Track [21] was the first academic PCG competition. Gen-

erators received gameplay metrics from a test level and had 60 seconds to produce a personalised level. Six entries competed; evaluation used pairwise forced-choice ("which was more fun?") with 15 human judges. This competition established several norms that influenced subsequent research: pairwise comparison as the preferred human evaluation protocol, player telemetry as input to adaptive generators, and the A* agent as a *de facto* playability oracle.

Levels are stored as 2D character grids (ASCII tilemaps), where each character encodes a functional tile type. The Video Game Level Corpus (VGLC) [29] standardised this encoding across all 32 original SMB levels and has been widely used as training data for neural generators.

2.3.2 Mario Level Generators in the Literature

The PCG Arena platform includes 15 generators spanning all major paradigm families, selected to benchmark a broad cross-section of established PCG approaches. Table 2.1 provides a comparative overview.

KOT(gdzieś bym jeszcze wspomniał, nie wiem jakie miejsce jest dobre, że nie wszyscy potrafią generować to samo i że wiele z zaprezentowanych podejśc upraszcza sobie życie generując tylko 1-2 rodzaje przeciwników, albo tylko kilka rodzajów bloków. Swoja droga pytanie czy to nie jest issue że o samej grze i co składa się na level dowiadujemy się tak późno w pracy...)

An important caveat is that these generators do not all target the same design vocabulary. Several reduce the problem space substantially—for instance, restricting themselves to one or two enemy types (typically Goombas and Green Koopas) or omitting power-ups, pipes carrying Piranha Plants, or Bullet Bill cannons. This unevenness affects fair comparison: a generator producing only a subset of SMB's design elements may appear competitive on some metrics while being structurally less expressive than the original Nintendo levels.

Human-authored baseline. The 15 original SMB levels from Nintendo (1985), transcribed into the Mario AI Framework's format. These serve as the quality ceiling against which all algorithmic generators are measured.

Constructive generators. Four generators build levels left-to-right using probabilistic placement: the *Notch* generator shipped with Infinite Mario Bros, its *Parameterized* variant (NotchParam) exposing six tunable difficulty/style knobs [21], a *Randomized* variant (NotchParamRand) that randomly samples parameters per level, and *Hopper*, which alternates generated and pre-designed segments with adaptive probability adjustment.

Chunk-based assembly. *Occupancy-Regulated Extension* (ORE) [13] KOT(tu i w kolejnych akapitach brakuje mi 'that' do spoójności przedstawiania co robią generatory względem wcześniejszego tekstu) is a method that builds

levels by iteratively attaching hand-authored chunks at compatible anchor points, producing structurally distinctive levels.

Search-based generators. The *Grammatical Evolution* (GE) generator [23] evolves level-construction programs via a context-free grammar. Three *Pattern-based* generators [5] represent levels as sequences of micro-patterns extracted from original SMB levels and use evolutionary search with different fitness functions: pattern count (maximising motif frequency), pattern occurrence (maximising motif diversity), and weighted pattern count (rewarding rare motifs).

Neural generators. *MarioGAN* [33] combines a DCGAN with CMA-ES latent-space search. *MarioGPT* [28] uses a fine-tuned DistilGPT-2 with text-prompt control. *MarioDiffusion* [19] generates tile scenes from natural-language captions via a diffusion model. Finally, *MarioDPO*—this thesis's contribution—fine-tunes a GPT-2-based generator with Direct Preference Optimisation using PCG Arena votes.

Generator	Paradigm	Key technique	Control	Year
Original SMB1	Human	Manual design	—	1985
Notch	Constructive	Probabilistic L-to-R	None	2008
NotchParam	Constructive	Probabilistic + 6 params	Param. knobs	2011
NotchParamRand	Constructive	Probabilistic + random	Random	2011
Hopper	Constructive	Adaptive probabilistic	Adaptive	2010
ORE	Chunk-based	Anchor-point assembly	None	2010
GE	Search	Grammatical evolution	Via grammar	2012
Pattern Count	Search	Evolutionary (pattern freq.)	Fitness wts.	2014
Pattern Occurrence	Search	Evolutionary (unique patt.)	Fitness wts.	2014
Pattern Wt. Count	Search	Evolutionary (rarity-wt.)	Fitness wts.	2014
MarioGAN	ML (GAN)	DCGAN + CMA-ES latent	Fitness fn.	2018
MarioGPT	ML (LLM)	Fine-tuned DistilGPT-2	Text prompts	2023
MarioDiffusion	ML (Diff.)	Text-conditioned UNet	Text captions	2025
MarioDPO	ML + DPO	GPT-2 + pref. alignment	Text + DPO	2026

Table 2.1: Generators included in PCG Arena. All except MarioDPO (this thesis) are previously published or reference implementations.

By including generators from all paradigm families and spanning nearly four decades of game content creation, PCG Arena tests whether human preference data can discriminate generator quality more reliably than automated metrics.

2.4 Evaluation Methods for Procedural Content

Evaluation is the central methodological challenge in PCG research and the primary motivation for PCG Arena. This section surveys the main families of evaluation methods used in the Mario PCG literature.

2.4.1 Automated Metrics

The most widely used evaluation approach computes *static structural metrics* directly from the tile grid, without simulating gameplay.

Linearity measures how closely a level's vertical profile follows a straight line. Smith and Whitehead [25] defined it as the mean absolute residual—the average vertical distance between platform midpoints and the best-fit regression line—so that lower values indicate a more linear level. Later works [9, 30] instead use R^2 `KOT(co to za wzorek, po co te literki i nazwy, to nic nie mowi, nic nie tłumaczy)` (the coefficient of determination, measuring the proportion of variance in the data explained by the regression), which inverts the scale: higher R^2 means more linear. The two formulations are therefore not directly comparable, complicating cross-paper analyses.

Leniency approximates difficulty through weighted sums of game objects. In its original form [25], it is computed as $L = (w_p \cdot n_p + w_g \cdot n_g + w_e \cdot n_e)/W$, where n_p is the number of power-ups, n_g the number of gaps, n_e the number of enemies, W the level width in tiles, and the weights $w_p > 0$, $w_g, w_e < 0$ reward forgiving content and penalise hazards. At least three incompatible formulations exist in the literature, using different tile weights and normalisation schemes [25, 23, 12]. Critically, Mariño et al. [12] showed that leniency only weakly correlates with human-rated difficulty.

Compression distance (gzip-based Normalised Compression Distance) quantifies structural similarity between level pairs and is used as a diversity proxy [23, 9].

Expressive Range Analysis (ERA), introduced by Smith and Whitehead [25], generates a large sample of levels, computes two metrics per level (classically linearity vs. leniency), and visualises the joint distribution as a 2D histogram, revealing a generator's output peaks, holes, and biases. ERA has been widely adopted [9, 23] but is purely descriptive—it characterises *what* a generator produces, not whether players enjoy it.

Figure 2.1 shows an ERA grid for the 13 PCG Arena generators, computed over 100 levels per generator (15 for the original SMB1 baseline). Each panel is a 2D histogram of linearity (using the R^2 formulation) against leniency. The plot reveals each generator's expressive range: e.g. Notch and its variants concentrate near high-linearity, low-hazard regions, while pattern-based and learned generators populate broader regions of the design space.

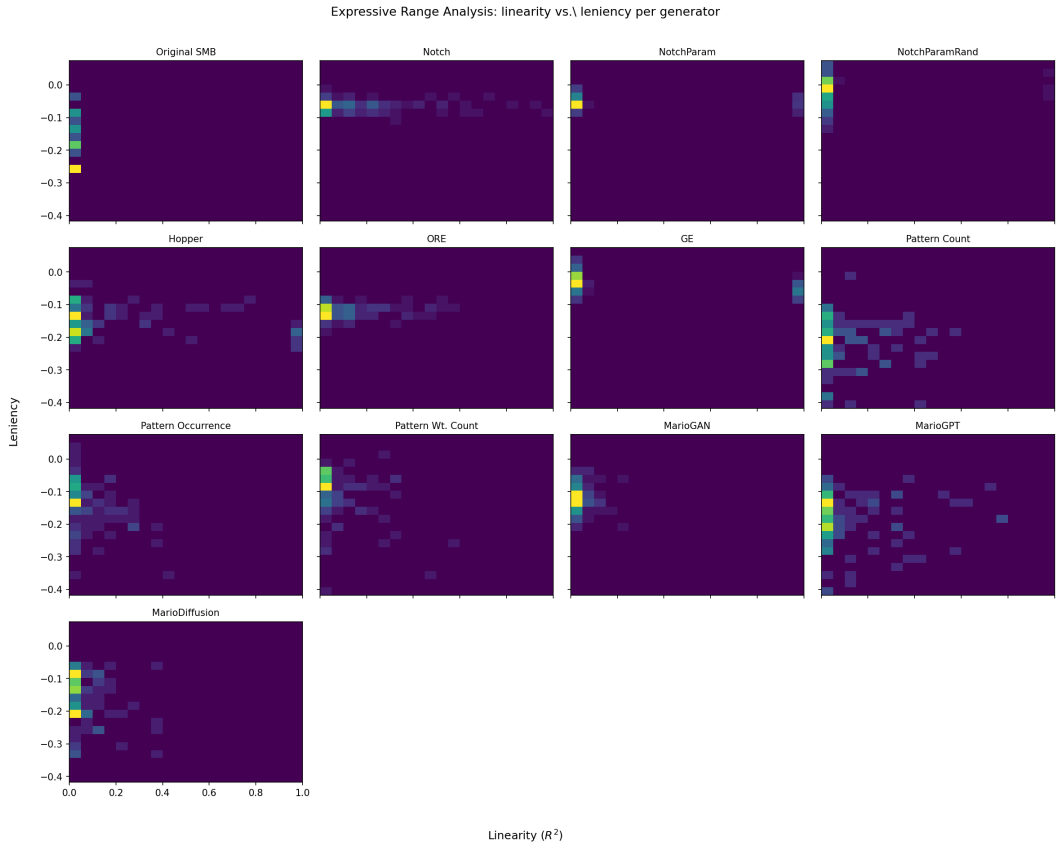


Figure 2.1: Expressive Range Analysis grid: 2D histogram of linearity (R^2) versus leniency for each PCG Arena generator. Each panel uses the same axes; brighter cells indicate more levels at that combination.

2.4.2 Agent-Based Testing

AI agents serving as player proxies enable scalable, repeatable assessment of playability and difficulty.

Playability gates. A level is deemed playable if a strong A* agent can complete it. This binary check is the most common first stage in PCG evaluation pipelines [33, 29, 11].

Difficulty proxies. Agent-derived metrics include jump count (the number of jumps the A* agent performs, interpreted as difficulty), completion fraction (progress along the x-axis if the level is not fully completable), and virtual damage from stochastic "human-like" agents that model input timing inaccuracies [14].

Agent-based evaluation is fast, reproducible, and incurs no participant cost, but agent bias is a fundamental concern: superhuman agents can solve levels humans cannot, and difficulty estimates depend on the specific agent policy. The mismatch also goes the other way: many levels that human players can complete are unsolvable by standard A* agents, which lack the ability to backtrack, use special power-ups, or

exploit advanced movement techniques. These bi-directional limitations are rarely quantified in the literature. KOT(czytam podobny tekst już 2 czy trzeci raz w tej pracy ten tekst i brakuje mi, tu lub wcześniej, przeciwwagi: że jest masa leweli przechodzalnych przez ludzi którzych te używane AI nie potrafią - głównie bo backtracking i special skille)

2.4.3 Human Evaluation Studies

Human studies remain the gold standard for PCG evaluation but vary widely in scale and methodology.

Pairwise forced-choice. The Mario AI Championship [21] asked 15 judges to play two levels and select "which was more fun?" without further defining "fun." This protocol directly inspired PCG Arena's design.

Likert-scale ratings. Mariño et al. [12] used 7-point Likert scales (ordered response scales from "strongly disagree" to "strongly agree") for enjoyment, aesthetics, and difficulty with $n = 37$ participants.KOT(wtf is n?) Summerville et al. [30] collected ratings across 85 design metrics and showed that inter-rater agreement imposes ceilings on achievable metric–human correlations. The practical consequence is a quantitative upper bound: an automated metric cannot correlate with human ratings any better than humans correlate with each other, so reporting an inter-rater agreement baseline is essential when interpreting metric–human correlations—a discipline rarely followed in the PCG literature. KOT(also - what is likert scale...)

Large-scale web studies. Pedersen et al. [16] collected 240 game pairs via a web deployment of *Infinite Mario Bros* variants for pairwise affective-preference analysis. A follow-up by Shaker et al. [22] collected 600 game-pair comparisons via an uncontrolled web applet, but the number of unique participants is not reported. Both studies date from 2010–2011 and predate the wave of ML-based generators (GANs, LSTMs, diffusion models) that have since reshaped the field.

These studies collectively demonstrate that while human empirical evaluation provides the most valid quality signal, it is expensive, typically small-scale ($n < 40$ in controlled settings), and conducted as isolated, one-time events rather than within a persistent evaluation infrastructure. Earlier work on automatic personalised content generation [20] and design-pattern analysis [4] further explored the relationship between level structure and player preferences, but likewise relied on small participant pools. Moreover, even the largest prior web study was conducted before modern generative methods existed, leaving a significant gap in comparative human evaluation of contemporary Mario level generators.

KOT(więcej tego jeszcze było z human studies: https://ojs.aaai.org/index.php/AIID oraz http://julian.togelius.com/Dahlskog2013Patterns.pdf ze slajdów złapałem

co mi się przypomniało)

2.4.4 Comparison of Approaches and Their Limitations

Table 2.2 summarises the evaluation methods employed across key papers in the Mario PCG literature.

Paper	Year	PCG algorithm	S	A	ERA	H	ML
Smith & Whitehead	2010	Launchpad (rhythm-based)	✓		✓		
Pedersen et al.	2010	Notch (parameterised)	✓			✓	✓
Shaker et al. (Champ.)	2011	Comp. entries (mixed)	✓	✓		✓	
Shaker et al. (GE)	2012	Grammatical Evolution	✓		✓		
Horn et al.	2014	Multiple (comparative)	✓		✓		
Marino et al.	2015	GE + Notch (compared)	✓			✓	
Summerville & Mateas	2016	LSTM column generator	✓	✓			✓
Summerville et al.	2017	Multiple (8 generators)	✓	✓		✓	✓
Volz et al. (MarioGAN)	2018	DCGAN + CMA-ES	✓	✓			
Nam et al.	2024	Stochastic "human-like" agents	✓	✓		✓	
PCG Arena (ours)	**2026**	**15 generators (all paradigms)**	✓	✓	✓	✓	✓

Table 2.2: Evaluation methods used in Mario PCG research. S = structural metrics; A = agent-based; ERA = Expressive Range Analysis; H = human study; ML = learned predictor.

Taken together, the works summarised in Table 2.2 illustrate how Mario PCG evaluation has progressed: from purely structural analyses, through agent-based playability checks and expressive-range characterisations, to small-scale human studies and learned predictors of player behaviour. Across this body of work, three persistent problems emerge:

Metrics–experience misalignment. Mariño et al. [12] showed that two generators rated identically by all computational metrics were rated *significantly differently* by human players on enjoyment. Summerville et al. [30] further demonstrated that inter-rater agreement imposes ceilings on achievable metric–human correlations, with aesthetics being particularly noisy. Automated metrics are useful characterisation tools but unreliable quality proxies.

Small-n and one-time studies. Most human studies use 15–37 participants in controlled settings and were conducted as isolated events, often years apart. No prior platform is known to offer persistent, longitudinal evaluation at scale—when players know which algorithm produced a level, expectations contaminate judgments.

Absence of a community-standard reward model. Search-based and RL-based generators require fitness or reward functions, yet no validated reward model for Mario level quality exists. The training data needed to build such a model—

large-scale human preferences over diverse generators—has been unavailable.

PCG Arena addresses these gaps by providing a persistent, blind, web-based platform for human evaluation, producing the preference data needed to (a) rank generators with statistical confidence, (b) validate automated metrics—identifying which, if any, can serve as reliable proxies for human judgment and thereby reduce the cost of future evaluations—and (c) train preference-aligned generators.

2.5 Rating Systems for Pairwise Comparisons

PCG Arena uses pairwise comparisons as its primary data collection protocol. This section reviews the mathematical foundations underlying the platform's rating engine.

2.5.1 The Elo Rating System

All modern skill-rating systems rest on the **Bradley-Terry model** [1]: each item i is assigned a strength parameter π_i, and the probability that item i is preferred to item j is

$$P(i \succ j) = \frac{\pi_i}{\pi_i + \pi_j}. \tag{2.1}$$

Elo [6] operationalised this model for competitive chess. Each player carries a scalar rating R; the expected score of player A against player B is

$$E_A = \frac{1}{1 + 10^{(R_B - R_A)/400}}. \tag{2.2}$$

After an observed outcome $S_A \in \{0, 0.5, 1\}$, the rating updates as $R'_A = R_A + K(S_A - E_A)$, where K is a fixed gain constant. Elo's simplicity made it the standard in chess, tennis, and early online games, but its constant K cannot distinguish a well-established rating from a newcomer's, motivating Bayesian extensions.

2.5.2 The Glicko-2 Rating System

Glickman [7] recast Elo in a Bayesian framework (**Glicko**), modelling each player's rating as a Gaussian $\mathcal{N}(\mu, \sigma^2)$ where σ—the *rating deviation* (RD)—quantifies uncertainty. Between rating periods, RD grows to reflect increased uncertainty from inactivity.

Glicko-2 [8] extends Glicko with a third parameter, *volatility* σ_v, which captures how erratic an entity's performance is. The update pipeline converts ratings to an internal scale, estimates variance and improvement from outcomes, iteratively updates volatility, and then updates RD and rating. KOT(`chyba coś nie jest wprowadzone`

- czym jest phi?)KOT(ok, zobaczyłem potem ranking deviation wprowadzając gaussiana dałeś jako sigmę square a teraz używasz phi...) On the internal Glicko-2 scale the rating deviation is denoted $\phi = \mathrm{RD}/\lambda$, where $\lambda = 173.7178$ is the scale conversion factor:

$$\phi' = \frac{1}{\sqrt{1/\phi^{*2} + 1/v}}, \quad \text{where } \phi^* = \sqrt{\phi^2 + \sigma_v'^2}, \tag{2.3}$$

$$\mu' \leftarrow \mu + \phi'^2 \sum_j g(\phi_j)(s_j - E_j), \tag{2.4}$$

where $g(\phi) = 1/\sqrt{1 + 3\phi^2/\pi^2}$ down-weights predictions involving uncertain opponents, s_j is the observed outcome, and E_j the expected outcome.

PCG Arena uses Glicko-2 with generators and levels treated as "players": each human vote corresponds to a match outcome. RD provides a built-in measure of rating reliability, and volatility accommodates generators whose perceived quality shifts as new levels are added. Implementation details and parameter choices are described in Section 3.6.

2.5.3 Arena-Style Evaluation Platforms

The **arena paradigm**—collecting anonymous pairwise votes from a crowd to rank competing systems—has recently been adopted beyond games.

Chatbot Arena. Chiang et al. [2] deployed a web platform where users chat simultaneously with two anonymous LLMs and vote for the better response. Over 240,000 votes from 90,000+ users have been collected; rankings are computed via Bradley-Terry MLE. An active sampling strategy concentrates votes on close model pairs, mirroring the information-gain logic that motivates PCG Arena's AGIS algorithm. Chatbot Arena is the closest structural analogue to PCG Arena outside the games domain.

LLM-as-a-Judge. Zheng et al. [35] studied using GPT-4 as a surrogate judge for pairwise evaluation, achieving >80% agreement with human preferences. PCG Arena's *Judge Function* plays an analogous role: a lightweight model predicting which of two Mario levels a human would prefer.

Sarkar and Cooper [18] applied Glicko-2 to the human computation game *Paradox*, treating puzzle-tasks as rated entities alongside players. This is the most direct precedent for PCG Arena's use of Glicko-2 to rate game content rather than human players.

Snow et al. [26] showed that aggregating labels from multiple non-expert annotators can match expert-level quality, underpinning the design assumption that a sufficient number of non-expert pairwise votes, properly aggregated, yields a trustworthy ranking.

2.6 Preference Learning and Alignment

PCG Arena's preference data enables not only passive ranking but also active generator improvement via preference learning. This section reviews the two key paradigms: Reinforcement Learning from Human Feedback (RLHF) and its simplified successor, Direct Preference Optimisation (DPO).

2.6.1 Reinforcement Learning from Human Feedback

RLHF trains a policy to maximise a reward signal derived from human preferences rather than a hand-designed reward function. The pipeline crystallised over a sequence of four landmark papers:

1. Christiano et al. [3] proposed learning a reward model $\hat{r}$ from pairwise trajectory comparisons and optimising the policy against $\hat{r}$ via PPO. Applied to Atari and MuJoCo.
2. Ziegler et al. [36] adapted the scheme to language models (GPT-2) and introduced a KL penalty $\beta\,\mathrm{KL}[\pi\|\pi_{\text{ref}}]$ to prevent drift from the pretrained model.
3. Stiennon et al. [27] scaled to summarisation (6.7B model, 64K comparisons) and showed that the learned reward predicts human preferences better than ROUGE`KOT(co, kto, z czym w ogóle o co chodzi>)` (Recall-Oriented Understudy for Gisting Evaluation, a standard automatic metric for text summarisation based on n-gram overlap).
4. Ouyang et al. [15] codified the three-step recipe (SFT → Reward Model → PPO). A 1.3B InstructGPT was preferred over 175B GPT-3, demonstrating that alignment quality matters more than model scale.

The RLHF pipeline addresses the same fundamental problem as PCG Arena: *metric mismatch*. Just as ROUGE `KOT(ke? odnosisz sie do czegoś co wie ktokolwiek kto czyta tą pracę a nie rozmawia z tym samym LLMem?)` fails to capture summary quality [27], automated PCG metrics fail to capture fun, aesthetics, or challenge appropriateness [12]. PCG Arena's paired-comparison voting is the game-design analogue of RLHF's preference collection phase. `KOT(czyli wiemy, że jest jakiś pieline, który ustabilizował się w jakichś 4 papirach które oferuyją masę buzzwordów. czyli nie wiemy nic. Jakbyś obciął tą całą subsekcję po 1 zdaniu to właściwie content przayswajalny dla czytelnika zostałby bez zmian. Tak, jest późno, pisałem dzisiaj recenzje paperów i mój saltyness zaczyna znowu wzrastać...)`

2.6.2 Direct Preference Optimization

Rafailov et al. [17] observed that the optimal RLHF policy can be expressed in closed form as a function of the reward model. Substituting this solution back into the

reward-model loss yields **Direct Preference Optimisation** (DPO), which directly fine-tunes the policy using preference pairs—eliminating both the reward model and the RL loop:

$$\mathcal{L}_{\mathrm{DPO}}(\pi_\theta; \pi_{\mathrm{ref}}) = -\mathbb{E}_{(x,\, y_w,\, y_l)} \left[\log \sigma \left(\beta \log \frac{\pi_\theta(y_w|x)}{\pi_{\mathrm{ref}}(y_w|x)} - \beta \log \frac{\pi_\theta(y_l|x)}{\pi_{\mathrm{ref}}(y_l|x)} \right) \right] \tag{2.5}$$

where y_w and y_l are the preferred and dispreferred completions, β controls the deviation from the reference policy π_{ref}, and σ is the logistic sigmoid. The gradient implicitly increases the log-likelihood of preferred outputs and decreases that of dispreferred ones, weighted by how much the current policy already agrees. DPO matched or exceeded PPO-based RLHF on sentiment, summarisation, and dialogue tasks while being simpler to implement and more stable to train.

Application to PCG Arena. MarioDPO applies DPO to Mario level generation. Preference pairs are constructed from PCG Arena votes: the level receiving the human vote becomes y_w, the rejected level becomes y_l. The DPO objective fine-tunes a GPT-2-based level generator to produce levels more aligned with human preferences, closing the loop from human evaluation back to generation. The training procedure is detailed in Chapter 5.

Chapter 3

PCG Arena — System Design

This chapter presents the design and implementation of PCG Arena, a web-based platform for blind A/B testing of procedural content generators for Super Mario Bros. We describe the system architecture, game engine, battle and voting mechanics, the novel AGIS matchmaking algorithm, the Glicko-2 rating integration, the builder submission workflow, and the deployment infrastructure.

3.1 Requirements and Design Goals

The primary goal of PCG Arena is to enable rigorous, human-centered evaluation of PCG algorithms through pairwise comparisons. Translating this goal into a concrete system imposes several requirements:

1. **Blind comparison.** Players must not know which generator produced which level, eliminating brand-loyalty bias and ensuring votes reflect genuine quality perception.

2. **Browser-based gameplay.** No installation should be required; the full Super Mario Bros experience must run in the browser with faithful physics.

3. **Uncertainty-aware ranking.** The rating system must quantify confidence in each generator's ranking, not merely produce a point estimate.

4. **Efficient matchmaking.** With a limited player population, every battle should maximise information gain toward stable rankings.

5. **Open submission.** Any researcher should be able to submit their own generator and receive a rating, with minimal friction.

6. **Rich telemetry.** Beyond the binary vote, the platform should capture gameplay trajectories, qualitative tags, and timing data for downstream analysis.

3.2 System Architecture

PCG Arena follows a three-tier web architecture with clear separation of concerns: a *presentation layer* (browser-based frontend), an *application layer* (RESTful backend), and a *data layer* (relational database). Table 3.1 summarises the technology choices for each layer.

Layer	Technology	Rationale
Frontend	React 18 + TypeScript 5.6 (Vite)	Canvas rendering; type safety
Backend	Python 3.12, FastAPI	Auto OpenAPI docs; Pydantic validation
Database	SQLite	Single-file; ACID; zero-config
Proxy	Caddy	Automatic TLS via Let's Encrypt
Container	Docker + docker-compose	Reproducible deployment

Table 3.1: PCG Arena technology stack.

3.2.1 Frontend

The frontend is a single-page application built with React 18 and TypeScript 5.6, bundled by Vite. Its source tree is organised into five top-level directories:

```
frontend/src/
  api/ # Backend communication (fetch wrappers)
  engine/ # Mario game engine (ported from Java)
  components/ # Reusable React UI components
  pages/ # Full-page route components
  contexts/ # React context providers (auth, theme)
```

Game rendering uses the HTML5 Canvas API for direct pixel manipulation, while the rest of the UI is standard React. Vite provides sub-second hot-module replacement during development and produces an optimised static bundle for production.

3.2.2 Backend

The backend is implemented in Python using FastAPI, chosen for its automatic OpenAPI documentation generation, native `async/await` support, and built-in request validation through Pydantic models. Key modules include:

- `main.py` — route definitions and middleware (CORS, rate limiting)
- `auth.py` — Google OAuth 2.0 and email/password authentication
- `glicko2.py` — Glicko-2 rating algorithm
- `matchmaking.py` — AGIS matchmaking algorithm
- `builders.py` — generator submission and management

- `stats.py` — analytics and data export endpoints

The API is versioned under the `/v1/` prefix and exposes endpoints for battles, votes, statistics, authentication, builder operations, and admin management. Administrative endpoints additionally require either session-based admin access or an API key.

3.2.3 Database

SQLite was selected as the database engine due to its single-file storage (simplifying backups and deployment), ACID transaction guarantees, and zero-configuration operation. The schema is managed through 17 sequential SQL migrations and comprises the following tables:

Table	Purpose
`generators`	PCG algorithm metadata and ownership
`levels`	ASCII tilemap content with dimensions and content hash
`battles`	Pairwise comparison instances (left/right level pairing)
`votes`	User preference outcomes with telemetry and tags
`ratings`	Current Glicko-2 state (μ, RD, σ_vKOT`(sigma nie powinna mieć literki?)`) per
`rating_events`	Audit log of all rating changes
`generator_pair_stats`	Confusion matrix data (pairwise win/loss counts)
`users`	Authenticated user accounts
`user_sessions`	Active login sessions
`email_verifications`	Email verification tokens
`password_resets`	Password reset tokens
`player_profiles`	Anonymous player identities
`player_sessions`	Player session tracking
`level_stats`	Aggregate per-level gameplay statistics
`play_trajectories`	Detailed player position data for heatmaps
`level_features`	Structural feature extraction (tiles, enemies, gaps)
`schema_migrations`	Migration bookkeeping

Table 3.2: Database schema: all 17 tables.

3.3 Game Engine

A critical component of PCG Arena is the browser-based Super Mario Bros engine, which was ported from the Java-based Mario AI Framework [10] to TypeScript. The port required faithful preservation of gameplay physics while adapting rendering and input handling to browser constraints.

3.3.1 TypeScript Port

The porting process involved four main translation steps. KOT(jakaś notka o porvcie? na ile ręcznie na ile automatem jakimś, jak tak to jakim i czy były potrzebne jakieś fixy? Informacje o tym powinny się znaleźć w pracy) The port was carried out largely by hand, with assistance from AI-based coding tools for routine translation patterns. Ensuring faithful physics reproduction required iterative testing and manual corrections, particularly for collision detection, rendering synchronisation, and enemy-behaviour edge cases.

1. **Class structure.** Java classes were converted to TypeScript classes, preserving inheritance hierarchies (e.g. `MarioSprite` → `Mario`, `Enemy`).
2. **Physics constants.** All numerical constants were kept identical to the Java originals (Table 3.3).
3. **Rendering.** Java Swing/AWT rendering was replaced with the HTML5 Canvas API (`CanvasRenderingContext2D`).
4. **Input.** Keyboard event handling was reimplemented using browser `KeyboardEvent` listeners.

The key engine modules are:

```
engine/
  MarioGame.ts # Game loop (requestAnimationFrame)
  MarioWorld.ts # World state, collision, game logic
  MarioLevel.ts # ASCII tilemap parser
  MarioSprite.ts # Base sprite class (physics, rendering)
  sprites/ # Mario, enemies, items, projectiles
  graphics/ # Camera, sprite sheets
  effects/ # Particle effects
```

Parameter	Value
Ground inertia	0.89
Air inertia	0.89
Gravity	3.0
Jump velocity	−1.9
Walk speed	0.6
Run speed	1.2

Table 3.3: Core physics constants preserved from the Java Mario AI Framework.

The game loop runs at 30 ticks per second, matching the original Java framework. Rendering runs separately at 60 FPS via `requestAnimationFrame`, interpolating between physics frames. This decoupling preserves the exact physics behaviour

of the original framework (which assumes 30-tick updates) while providing visually smooth animation in the browser. Physics updates are fully deterministic, ensuring consistent gameplay across sessions.

3.3.2 Level Format

Levels are stored as plain-text ASCII tilemaps, where each character encodes a tile type or entity spawn point. The format supports 37 distinct tile characters, summarised in Table 3.4.

KOT(tego i w ogóle opisu samej gry, powerów też? etc brakuje wcześniejw backgroundzie, tak naprawdę nawet przed related Work, takie jest przynajmniej moje zdanie, nie wiem jak Sie Kindze czytało)

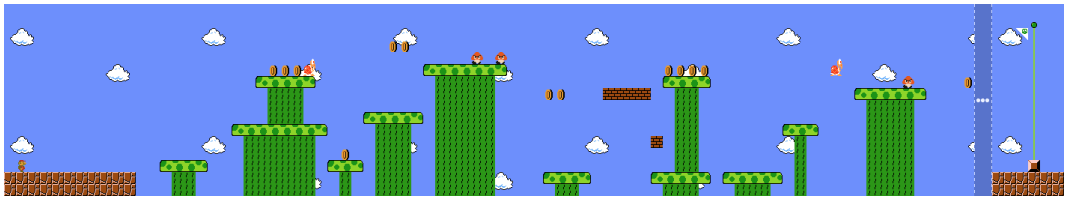
Char	Meaning	Char	Meaning
-	Air (empty)	Q	Question block (coin)
X	Solid floor	!	Question block (coin, alt)
S	Brick block	C	Coin block
#	Pyramid block	U	Mushroom block
D	Used block	L	1-Up block
%	Jump-through platform	1	Invisible 1-Up block
|	Platform background	2	Invisible coin block
?	Question block (mushroom)	o	Free-standing coin
@	Question block (mushroom, alt)	M	Mario spawn
E	Goomba	F	Flag (level exit)
g	Goomba (alt)	t	Pipe (empty)
G	Winged Goomba	T	Pipe (flower)
k	Green Koopa	<	Pipe top left
K	Winged Green Koopa	>	Pipe top right
r	Red Koopa	[	Pipe body left
R	Winged Red Koopa	]	Pipe body right
y	Spiky	*	Bullet Bill body
Y	Winged Spiky	B	Bullet Bill head
		b	Bullet Bill neck

Table 3.4: Complete ASCII tile legend (37 characters).

A typical level file consists of 16 rows (the default height) and up to several hundred columns, stored with newline-separated rows. For example, the original Super Mario Bros World 1-1 is encoded as a 202×16 character grid. Figure 3.1 shows a segment of this level. KOT(Jak dajesz ef do figure to już bez dwukropka. I BTW fajnie jakby obok była postawiona kolorowa wersja ;])

```
--------------------------------------------------------------------------------...------
--------------------------------------------------------------------------------...------
--------------------------------------------------------------------------------...------
--------------------------------oo------------------------------------------------...------
-----------------------------------------g-g--------------------------------------...------
----------------------ooor---------%%%%%%%-------------oooo----------R-----------...------
--------------------%%%%%----------|||||--------------%%%%----------------g----o...------
----------------------|||-----------|||||----oo---SSSS--||-------------%%%%%%----...------
----------------------|||-----------|||||----------------||--------------||||-----...------
----------------------|||-----%%%%%-|||||----------------||--------------||||-----...------
--------------------%%%%%%%%----|||--|||||----------------||-------%%%----||||-----...------
----------------------||||||-----|||--|||||-------------U-||--------|-----||||-----...------
----------------------||||||--o--|||--|||||----------------||--------|-----||||-----...---F--
-M----------%%%%---||||||-%%%-|||--|||||----------------||--------|-----||||-----...---#--
XXXXXXXXXXX---||----||||||--|--|||--|||||----%%%%-----%%%%%-%%%%%-|-----||||-----...XXXXXX
XXXXXXXXXXX---||----||||||--|--|||--|||||-----||-------|||---|||--|-----||||-----...XXXXXX
```

(a) ASCII tilemap.



(b) Rendered view of the same segment.

Figure 3.1: Original Super Mario Bros Level (World 1-1 segment). `M` marks Mario's spawn; `X` is solid ground; `U` and `S` are question blocks; `g` is a Goomba; `F` denotes level exit; `-` is empty air.

3.3.3 Level Validation

Every submitted level undergoes format validation before insertion into the database. The validation procedure, implemented in the backend's `seed.py` module, checks:

1. **Height bounds:** the number of rows must satisfy $10 \leq h \leq 20$.
2. **Width bounds:** the number of columns must satisfy $150 \leq w \leq 250$. In practice, all submitted generators produce levels with widths between 150 and 208 columns; KOT(??) the lower bound of 150 corresponds to the standard screen width of the original Super Mario Bros.
3. **Rectangular shape:** all rows must have identical length.
4. **Character whitelist:** every character must belong to the set of 37 allowed tiles (Table 3.4).

Levels that fail any check are rejected with a descriptive error message. Note that the platform performs *format-level* validation only; there is no automated playability

or reachability analysis (e.g. checking that the exit is reachable from the spawn). Content hashes are computed for deduplication.

3.4 Battle and Voting System

The core user experience is the *battle*: a blind A/B test in which the player compares two levels produced by different generators, not knowing which generator produced which level.

3.4.1 Battle Flow

A battle proceeds through five stages:

1. **Battle request.** The frontend calls `POST /v1/battles:next`.

2. **Matchmaking.** The AGIS algorithm (Section 3.5) selects two generators; one random level is drawn from each.

3. **Gameplay.** The player plays both levels sequentially (labelled "Left" and "Right"). During play, the engine records a trajectory of Mario's position and death events.

4. **Voting.** After completing both levels, the player chooses one of four options: *Left is Better*, *Right is Better*, *Tie*, or *Skip*. The "Skip" option, used when the player feels unable to judge or encounters a technical issue, does not affect ratings.

5. **Rating update.** The backend updates Glicko-2 ratings for both generators atomically within a database transaction (Section 3.6).

Figure 3.2 shows the main gameplay interface. After completing both levels, the voting panel (Figure 3.3) allows the player to cast their preference and optionally tag each level.

Figure 3.2: Main gameplay view: the player experiences a procedurally generated level in the browser-based Mario engine.

Figure 3.3: Voting panel shown after both levels are played, with preference buttons. Tag selection is accessible by hovering over the level (see Figure 3.4).KOT(tag selection part is not visible onthis screen actually)

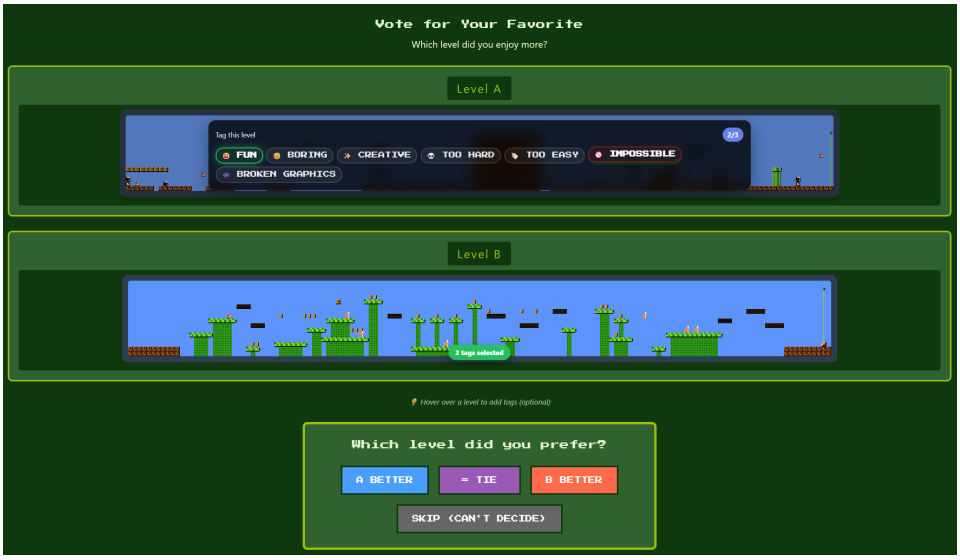


Figure 3.4: Tag selection popup shown when the player hovers over a level. Players may assign up to three descriptive labels per level.

3.4.2 Tagging System

In addition to the preference vote, players may optionally tag each level in the battle with up to three descriptive labels drawn from a fixed vocabulary of seven tags. The tags were hand-crafted based on properties a PCG researcher would want a generator to be judged on: overall enjoyment, engagement, creativity, difficulty calibration, playability, and visual correctness. We did not find a pre-existing tag taxonomy for PCG level evaluation in the literature; the set was designed to be small enough for rapid annotation yet expressive enough to capture the most salient quality dimensions.

Tag	Description
`fun`	The level was enjoyable to play
`boring`	The level lacked engagement
`creative`	The level showed novel or interesting design
`too_hard`	The level was excessively difficult
`too_easy`	The level was trivially easy
`impossible`	The level appeared to be uncompletable
`broken_graphics`	The level had visual or rendering issues

Table 3.5: The seven available tags for level characterisation.

Tags are stored as boolean columns in the `votes` table and aggregated in `level_stats`, enabling analyses such as correlating perceived difficulty with win rates or identifying generators whose levels are consistently tagged as `impossible`.

3.5 Matchmaking: AGIS Algorithm

3.5.1 Problem Definition

Given n generators with current Glicko-2 ratings and uncertainties, the matchmaking problem is to select a pair for the next battle that maximises information gain toward accurate final rankings. Naive strategies are suboptimal:

- **Uniform random** wastes battles on already-converged pairs.
- **Most-uncertain-first** neglects head-to-head comparisons between similarly rated generators.
- **Round-robin** ignores rating information and pairs dissimilar generators.

To address these shortcomings, we designed **AGIS (Adaptive Glicko-Informed Selection)**, a custom two-stage matchmaking algorithm that incorporates rating uncertainty, skill similarity, and pairwise coverage into generator selection. While the individual components draw on well-known ideas from information-theoretic experimental design (uncertainty sampling, coverage balancing), their combination into a lightweight, Glicko-2-aware matchmaker tailored to the PCG evaluation setting is, to our knowledge, novel.

3.5.2 Stage 1: First Generator Selection

Each generator i receives a selection weight w_i composing an uncertainty term and a games-played term:

$$w_i = \left(1 + \widetilde{\mathrm{RD}}_i\right)^2 \cdot \mathrm{boost}(\mathrm{games}_i) \tag{3.1}$$

where $\widetilde{\mathrm{RD}}_i = (\mathrm{RD}_i - \mathrm{RD}_{\min})/(\mathrm{RD}_{\max} - \mathrm{RD}_{\min})$ is the normalised rating deviation. The *boost*`KOT(boost też zmienionym fontem)` function provides a 4× weight multiplier for generators with zero games, linearly decaying to 1× at `MIN_GAMES` (default: 30). After convergence, a mild quality bias is applied:

$$\mathrm{boost}_{\mathrm{converged}} = 0.8 + q_{\mathrm{bias}} \cdot \min\left(1,\ \max\left(0,\ \tfrac{\mu_i - 600}{800}\right)\right) \tag{3.2}$$

where $q_{\mathrm{bias}} = 0.2$ is the configurable quality bias strength. The weights are normalised to sum to 1, forming a categorical (discrete) probability distribution; the first generator is then drawn from this distribution via weighted random sampling.

3.5.3 Stage 2: Second Generator Selection

The intuition behind Stage 2 is that the most informative battle is one between generators of *similar* strength (maximising discriminative power) whose ratings are still *uncertain* (maximising information gain per battle), while ensuring that every pair accumulates enough head-to-head data for a reliable confusion matrix. The four weighted components below formalise these intuitions.

Given the first generator g_1, the second is selected using a composite pair weight:

$$w_{1,j} = \alpha \cdot S(g_1, g_j) \; + \; \beta \cdot U(g_j) \; + \; \gamma \cdot I(g_1, g_j) \; + \; \mathrm{cov}(g_1, g_j) \tag{3.3}$$

The four components are:

- **Rating similarity** $S(g_1, g_j) = \exp\!\left(-\Delta\mu^2 / 2\sigma_s^2\right)$, a Gaussian kernel with $\sigma_s = 150$ penalising large rating gaps.
- **Uncertainty** $U(g_j) = 1 + \widetilde{\mathrm{RD}}_j$, favouring uncertain generators (range $[1, 2]$).
- **Information gain** $I(g_1, g_j)$, incorporating the joint RD and a match-quality estimate.
- **Coverage bonus** $\mathrm{cov}(g_1, g_j)$: for pairs with fewer than `TARGET_BATTLES_PER_PAIR` (default: 10) battles, an exponential bonus $2 \cdot e^{-c/3}$ is applied, where c is the current battle count; otherwise a residual 0.1 is used.

3.5.4 Parameters

Table 3.6 lists the configurable AGIS parameters and their defaults, which are set via environment variables.

Parameter	Description	Default
α	Rating similarity weight	0.5
β	Uncertainty weight	0.3
γ	Information gain weight	0.2
`MIN_GAMES`	Games before "converged"	30
`TARGET_BATTLES_PER_PAIR`	Min. battles per pair	10
`RATING_SIGMA`	Similarity kernel σ_s	150.0
`QUALITY_BIAS`	Quality bias strength q_{bias}	0.2

Table 3.6: AGIS algorithm parameters and default values.

3.6 Glicko-2 Rating System

PCG Arena uses the Glicko-2 rating system [8] to maintain uncertainty-aware rankings. Each generator carries three parameters:

- **Rating (μ):** the skill estimate, displayed on a scale centred at 1000.
- **Rating Deviation (RD, ϕ):** the uncertainty in the rating. High RD indicates an unreliable estimate; low RD indicates a well-established rating.
- **Volatility (σ):** the expected fluctuation in performance over time.

KOT(te dwie subsekcje to właściwie była kopia tego co było już wcześniej)

The mathematical foundations of the Glicko-2 system—the Bradley-Terry model, expected score computation, and the update equations for ϕ' and μ'—are presented in Section 2.5.2. Here we describe the implementation-specific aspects of how PCG Arena applies these formulas.

3.6.1 Rating Update Procedure

After each battle, the winning generator receives a score of 1, the loser 0, and ties yield 0.5 for both. Using the expected score and update equations from Section 2.5.2, the implementation proceeds as follows:

1. Convert ratings to the internal Glicko-2 scale (centred at 0, divided by $\lambda = 173.7178$). This constant equals $400/\ln 10 \approx 173.7178$ and bridges the Elo convention (400 points per tenfold win-probability change) with the natural logistic function used internally by Glicko-2.
2. Compute the variance v from the opponent's RD.
3. Compute $\Delta = v \cdot g(\phi_B)(s - E)$, where s is the actual score.
4. Update volatility σ' via the iterative Illinois algorithm (system constant $\tau = 0.5$).
5. Update RD and rating using Equations 2.3–2.4.
6. Convert back to the display scale and clamp to $[100, 3000]$.

All rating updates are performed atomically within a single database transaction, with an audit record written to the `rating_events` table.

3.6.2 Configuration

Table 3.7 lists the Glicko-2 constants used by PCG Arena.

Parameter	Value
Initial rating (μ_0)	1000
Initial RD (ϕ_0)	350
Initial volatility (σ_0)	0.06
System constant (τ)	0.5
Scale factor (λ)	173.7178
Minimum RD	30
Maximum RD	350
Minimum rating	100
Maximum rating	3000

Table 3.7: Glicko-2 configuration parameters.

The RD of each generator provides a natural 95% confidence interval for its true rating: $\mathrm{CI}_{95\%} = [\mu - 1.96\,\mathrm{RD},\ \mu + 1.96\,\mathrm{RD}]$. However, the public leaderboard displays only the point-estimate rating; confidence intervals are not shown on the platform's frontend. Figure 3.5 shows the leaderboard as seen by players.

Figure 3.5: Public leaderboard ranking generators by their Glicko-2 rating.

Clicking a generator reveals detailed statistics including rating history, per-opponent win rates, a level gallery, and tag distributions (Figure 3.6).

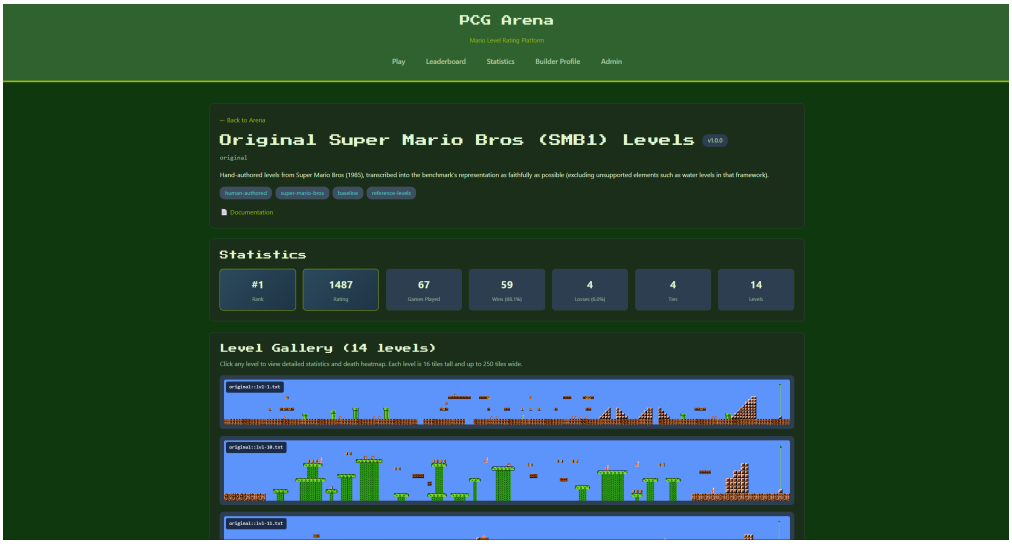


Figure 3.6: Generator detail page showing level gallery and performance statistics.

3.7 Builder Profile and Generator Submission

Authenticated users may submit their own generators through the Builder Profile page. The submission workflow is:

1. The user provides generator metadata: name, version, description, and a URL to the generator's source repository.
2. The user uploads a ZIP archive containing level files in the ASCII tilemap format.
3. The backend validates every level (Section 3.3.3). Levels that fail validation are rejected with detailed error messages.
4. Valid levels and the generator record are inserted into the database.
5. An initial Glicko-2 rating of 1000 ± 350 is assigned.
6. The generator appears on the leaderboard immediately and enters the matchmaking pool.

Each user may own at most three generators. When a user wishes to remove a generator, the system distinguishes two cases. If the generator has already participated in battles, it is *soft-deleted*: marked as inactive and excluded from future matchmaking, but its records (levels, ratings, votes) are retained so that historical analyses and the confusion matrix remain consistent. If the generator has never appeared in a battle, it is *hard-deleted* (fully removed from the database) since no other data depends on it.

Figure 3.7 shows the Builder Profile page. KOT(chyba tym wszystkim screenom by się przysłużył clip do środkowej meaningful części. Tzn ja lubię zielony, połacie zielonego po bokach mi nie przeszkadzają, ale jednak readability się pogarsza o nie pozwala trochę powiększyć obrazka żeby coś było widać na powiększeniu mniejszym niż 180)

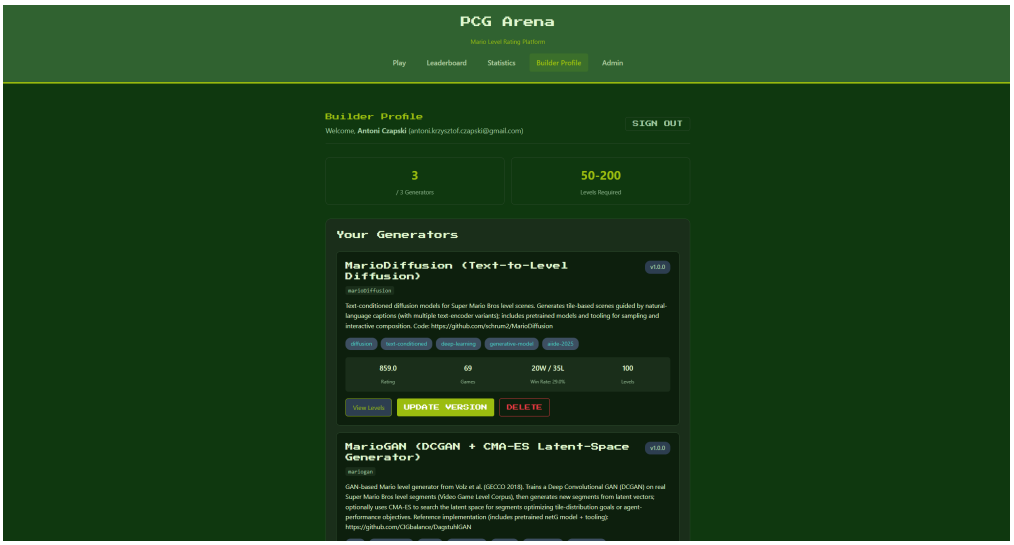


Figure 3.7: Builder Profile page for submitting and managing generators.

3.8 Authentication

PCG Arena supports two authentication methods:

- **Google OAuth 2.0.** The frontend opens a Google sign-in popup; the returned ID token (JWT) is sent to `POST /v1/auth/google`, where the backend verifies it against Google's servers. Google users are automatically marked as email-verified.

- **Email and password.** Users register with an email, password, and display name. Passwords are hashed with bcrypt (work factor 12). A verification email is sent via Twilio SendGrid containing a unique token (64 hex characters, 24-hour expiry).

Upon successful authentication, the backend issues a session cookie (`arena_session`) containing a 256-bit cryptographically random token. Sessions expire after seven days. HTTP-only cookies prevent client-side script access.

Administrative access is granted to users whose email addresses appear in the `ARENA_ADMIN_EMAILS` configuration list.

3.9 Deployment and Infrastructure

PCG Arena is deployed on Google Cloud Platform using a single Compute Engine instance (`e2-micro`, Ubuntu 22.04 LTS). The backend runs inside a Docker container orchestrated by `docker-compose`, with the SQLite database file bind-mounted from the host for persistence across container rebuilds.

Caddy serves as the reverse proxy, handling:`KOT(noindent)`

- Automatic TLS certificate provisioning and renewal (Let's Encrypt).
- HTTP-to-HTTPS redirection.
- Reverse proxying API requests to `localhost:8080`.
- Static file serving for the frontend production build.

The platform is accessible at `https://pcg-arena.com`. The backend binds to `127.0.0.1:8080` and is not exposed to the public internet directly; all external traffic passes through Caddy. Firewall rules allow only ports 80 and 443. Database backups are performed via a scheduled script that copies the SQLite file to a timestamped archive.

Chapter 4

User Study and Exploratory Data Analysis

This chapter presents the PCG Arena data as an exploratory user study and generator characterisation, organised around five research questions:

1. **RQ1**: Which generators are preferred by players in blind pairwise comparisons?
2. **RQ2**: How do generators differ in static expressive range, and which structural properties align with player preference?
3. **RQ3**: What patterns of player movement appear in the recorded trajectories for each generator?
4. **RQ4**: Are player preferences and play styles homogeneous, or do visible subgroups emerge?
5. **RQ5**: What do tags reveal about why levels win or lose?

The analysis is exploratory: its conclusions rely on patterns observed in the data and its visualisations rather than on formal hypothesis testing. The aim is to demonstrate what PCG Arena can measure—preference, structure, gameplay behaviour, user heterogeneity, and qualitative failure modes—while acknowledging that the current dataset is still too small to support claims of universal design laws.

4.1 Study Protocol and Dataset

Participants were recruited through university social-media channels, word of mouth among game-development and computer-science communities, and a small promoted Reddit campaign targeting game-development and procedural-generation communities. Participation was anonymous. The platform assigned browser-based anonymous player identifiers and session identifiers, persisted through cookies or local browser storage. Consequently, "players" in this analysis should be interpreted as anonymous browser/player IDs rather than verified unique humans; clearing cookies or changing

device/browser could create a new ID.

Each battle used a blind A/B protocol. A player saw two levels from different generators, played one attempt on each side, and then selected the preferred side, a tie, or skip. Optional tags could be attached to either side. Restricting each side to a single attempt was a deliberate system-design decision. A protocol allowing multiple attempts per level was also implemented and tested, but limiting play to one attempt per side collects substantially more pairwise comparisons per unit of player time, directly serving the platform's main goal of building a leaderboard from a large number of A/B comparisons. For interpretation, this means the death variable is a binary failure indicator for a single side-level attempt rather than a repeated-death count.

The May 2026 analysis export contains the data summarized in Table 4.1. Skip votes are included in engagement summaries but excluded from preference rankings. Ties are treated as 0.5 points for each side.

Table 4.1: Dataset summary for the exploratory analysis. Player counts refer to anonymous browser/player IDs, not verified unique humans.

Quantity	**Count**
Vote rows analyzed	1,000
Anonymous player profiles	79
Anonymous player IDs in vote table	77
Session IDs in vote table	213
Level-stat records exported	1,104
Generator IDs in research plots	14
Per-side telemetry records	2,000
Played side records	1,968
Embedded trajectory records	1,968
Tag assignments	329

The vote outcomes are balanced between the two screen sides (418 left wins and 398 right wins), with 165 ties and 19 skips, indicating no systematic side bias; moreover, the assignment of levels to the left or right side was randomised, further reducing the risk of positional bias. The median play duration is 8.0 seconds, but a small number of inactive-tab outliers are present; duration-dependent plots therefore use clipping or robust summaries.

4.2 RQ1: Generator Preference Ranking

The core purpose of PCG Arena is to measure human preference between generators. For each non-skip battle, the selected side receives score 1, the rejected side receives

score 0, and ties give 0.5 to both sides. Figure 4.1 shows the resulting generator leaderboard with bootstrap uncertainty intervals over votes.

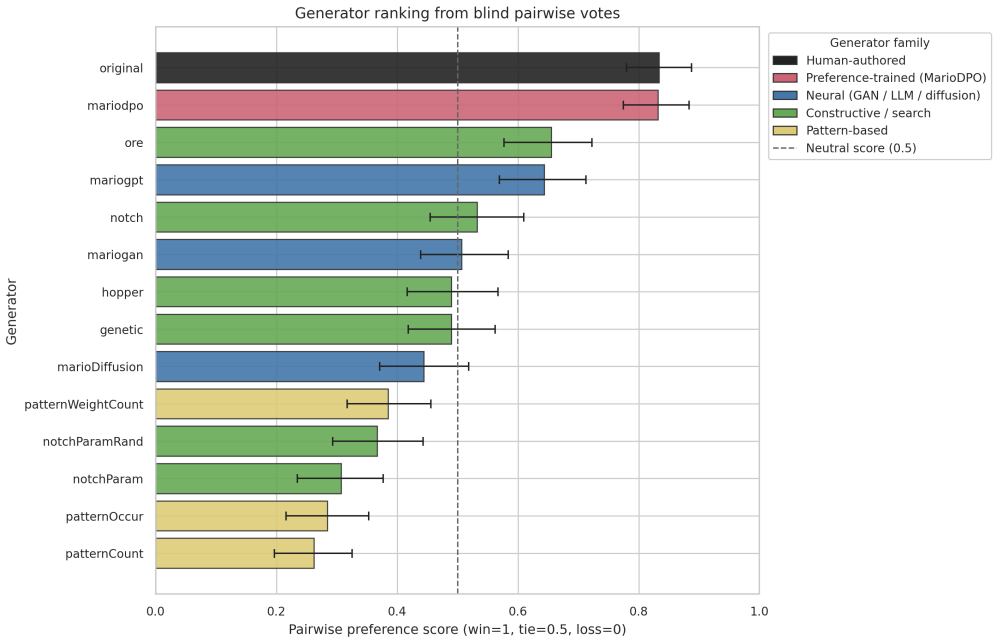


Figure 4.1: Generator ranking from blind pairwise votes. Bars show mean preference score, where win = 1, tie = 0.5, and loss = 0; horizontal intervals show bootstrap uncertainty. Bar colour encodes the generator family (human-authored, preference-trained, neural, constructive/search, and pattern-based), as shown in the in-plot legend.

The strongest result is that the human-authored `original` baseline and `mariodpo` are nearly tied at the top. `original` scores 0.833, while `mariodpo` scores 0.832. Both are well above the neutral 0.5 line. The next tier consists of `ore` (0.655) and `mariogpt` (0.643), followed by mid-table constructive and neural generators. The pattern-based generators occupy the bottom of the ranking: `patternWeightCount` scores 0.385, `patternOccur` scores 0.284, and `patternCount` scores 0.262.

This ordering is meaningful for two reasons. First, the human-authored baseline remains a strong benchmark, which is expected if the vote data captures level quality. Second, `mariodpo` approaches the human baseline and clearly outperforms the other procedural generators in this export, suggesting that preference-aligned generation is a promising direction for the platform.

Figure 4.2 gives the corresponding head-to-head matrix. Rows and columns are ordered by the ranking above. The matrix reveals that the top generators are not benefiting only from weak opponents: `original` and `mariodpo` perform well across many pairings, whereas pattern generators lose consistently across opponents.

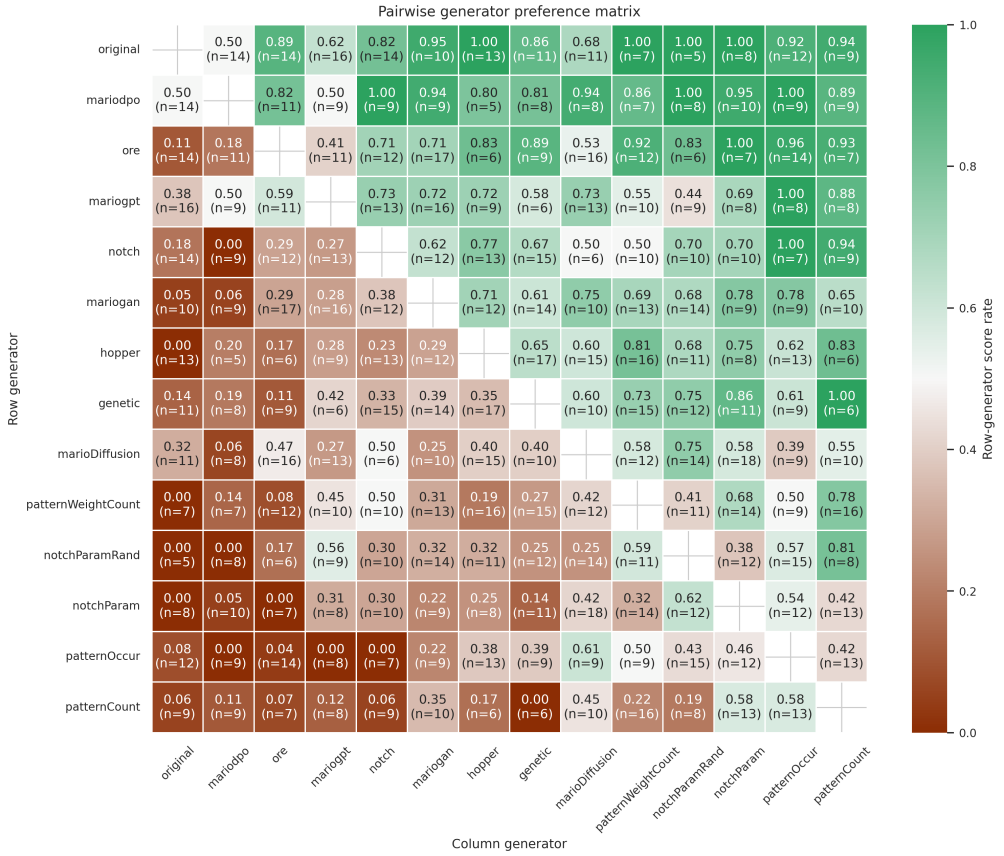


Figure 4.2: Pairwise generator preference matrix ordered by overall ranking. Each cell is the score rate for the row generator against the column generator, with ties counted as 0.5. Green indicates a higher score rate for the row generator.

4.3 RQ2: Static Expressive Range

The second research question asks what we can learn about a generator by looking at the levels themselves, without involving any player. The input is a level as plain text (an ASCII tilemap), and the output is a small set of numbers describing what the level looks like: how flat or hilly it is, how many enemies it contains, how often it has gaps, how many rewards it offers, and how similar its columns are to one another. Computing these numbers across all generators makes it possible to compare them on a common footing and to check whether any of these structural properties line up with the player preference scores from RQ1.

We compute the metrics directly from the ASCII level files. Linearity is the coefficient of determination (R^2) of a least-squares line fitted to the playable surface profile; it measures how well a straight line explains the surface, so a higher R^2 corresponds to flatter, more linear and more predictable terrain, while a lower value indicates a bumpier, more irregular profile. Density metrics count solid, enemy, reward, and gap structure per tile or per level column. Leniency combines gap density,

enemy density, maximum gap width, rewards, and blocks into a single descriptive score; because the various leniency definitions in the PCG literature are not mutually comparable, we use our formulation only to compare generators within this study, not as an absolute, cross-paper measure of difficulty. Diversity is estimated with tile/column entropy, the unique-column ratio, the compression ratio, and gzip normalized compression distance (NCD).

Figure 4.3 summarizes the main static metrics by generator. Figure 4.4 then shows each generator's expressive range in the plane of linearity and operational leniency.

Generator	Score	Levels	Linearity	Leniency	Gap dens.	Enemy dens.	NCD
original	0.83	15	0.02	0.61	0.18	0.08	0.83
mariodpo	0.83	200	0.06	0.67	0.14	0.09	0.81
ore	0.65	100	0.11	0.63	0.16	0.06	0.85
mariogpt	0.64	100	0.10	0.55	0.22	0.06	0.76
notch	0.53	100	0.12	0.70	0.10	0.02	0.75
mariogan	0.51	100	0.04	0.62	0.10	0.07	0.73
hopper	0.49	100	0.17	0.63	0.11	0.12	0.73
genetic	0.49	100	0.03	0.73	0.05	0.03	0.77
marioDiffusion	0.44	100	0.05	0.44	0.28	0.02	0.78
patternWeightCount	0.38	100	0.07	0.72	0.08	0.13	0.87
notchParamRand	0.37	100	0.14	0.74	0.07	0.02	0.73
notchParam	0.31	100	0.05	0.71	0.07	0.02	0.70
patternOccur	0.28	100	0.05	0.70	0.08	0.17	0.87
patternCount	0.26	100	0.15	0.72	0.05	0.25	0.83

Figure 4.3: Static expressive metrics computed from level text files. Metrics include fitted-surface linearity, operational leniency, gap density, enemy density, and within-generator compression distance. Each of the five metric columns is shaded within the column from red (low) to green (high) as a visual reading aid.

Figure 4.4: Expressive-range view of static level structure. Each subplot shows levels from one generator in the linearity–leniency plane, ordered by preference ranking.

Taken together, the two figures show several consistent patterns. The high-ranked generators `original`, `mariodpo`, `ore` and `mariogpt` are the only ones that combine non-trivial reward density with a moderate amount of gaps and enemies: there is something to collect, and there is some, but not too much, danger. The pattern-based generators sit at the opposite extreme. They produce zero rewards and the highest enemy density in the corpus, and on the expressive-range scatter their levels collapse into a narrow strip dominated by gaps and obstacles. Even when their leniency score looks moderate, this is a side-effect of having very few power-ups and little structural variety rather than evidence of a well-balanced design. Constructive generators such as `notch`, `notchParam` and `notchParamRand` lie in between: they offer flat, easy ground with low enemy density, but essentially no rewards, which is consistent with them being safe but bland. Finally, `ore` stands out as the most structurally diverse generator, with the highest unique-column ratio and the highest within-generator NCD, which helps explain why it climbs into the top tier despite using a much simpler chunk-based assembly approach than the neural generators.

Figure 4.5 relates selected structural summaries to preference score. With only 14 generators, these plots should be read as showing the direction and rough strength of each association, not as formal statistical tests. Reward density and unique-column ratio show the clearest positive association with preference score in this export, while distance from the `original` corpus by NCD is negatively associated with preference. Crucially, however, no single metric explains the ranking on its own: the better predictors still have visible exceptions, and difficulty proxies such as enemy density show no consistent relationship with preference at all. This is precisely the difficulty that motivates PCG Arena—player preference is a multi-dimensional signal that no individual static metric captures, which is why direct human comparison, rather than any automated proxy, is needed to rank generators reliably.

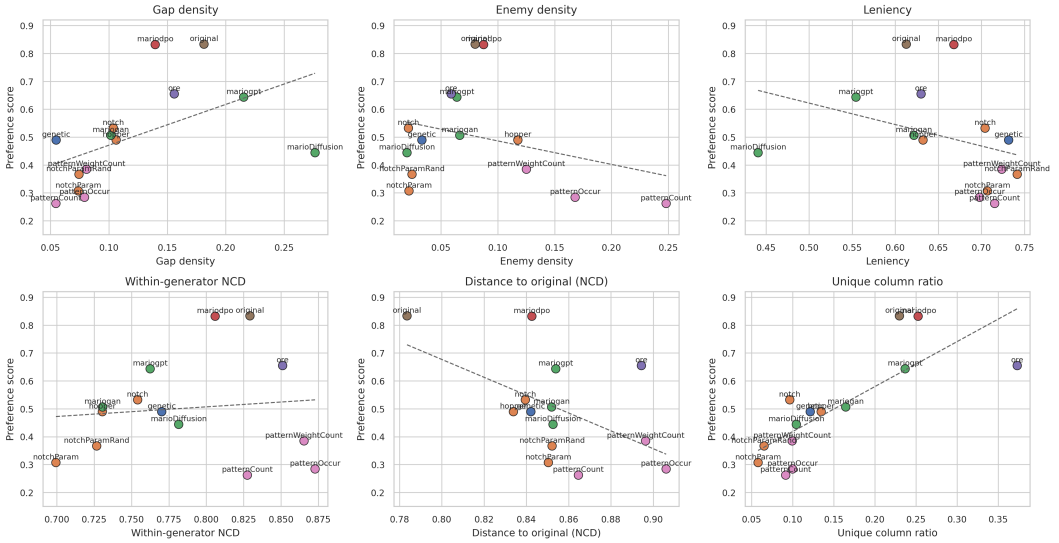


Figure 4.5: Generator preference score against selected static metrics. The fitted lines are descriptive trend guides, not causal claims.

4.4 RQ3: Gameplay Trajectory Fingerprints

For each generator, Figure 4.6 overlays all recorded Mario paths as thin red traces. This plot is intentionally simple: it shows where players actually moved in levels from each generator under the one-attempt battle protocol.

Figure 4.6: Stacked gameplay trajectories by generator. Each subplot overlays all recorded trajectories for one generator as thin red traces on a white background, ordered by preference score. The vertical axis is inverted to match screen coordinates.

The trajectory stacks make generator differences visually apparent. Stronger generators such as `original` and `mariodpo` show paths that often progress substantially through the level and occupy a broad portion of the play space. Several low-ranked pattern generators show many traces terminating early or concentrating in narrow regions, consistent with the high early-failure rates computed from death locations.

Figure 4.7 reduces these visual patterns to generator-level summaries. Occupancy entropy measures how broadly trajectories cover a fixed spatial grid. Median maximum x measures progress through the level. Verticality is the mean within-trajectory standard deviation of y. Death concentration is the largest share of death locations falling into a single horizontal bin. These are behavioral descriptors of a one-attempt protocol: death-location metrics indicate where failures happen, not how often players retry the same level.

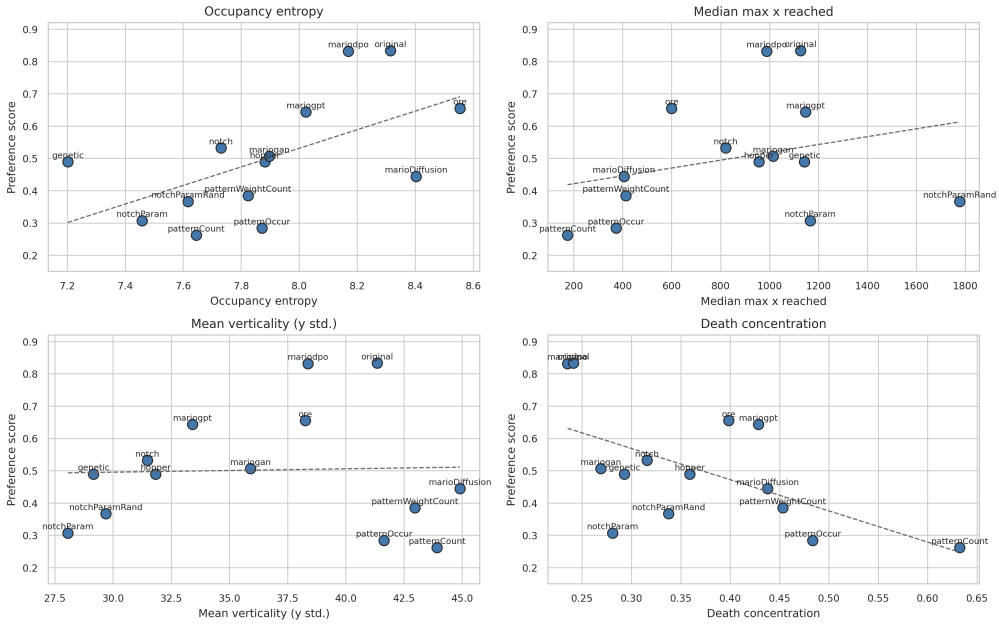


Figure 4.7: Trajectory-derived behavioral summaries versus generator preference score. Higher occupancy entropy and lower death concentration are associated with stronger generators in this export.

The strongest trajectory-level patterns are occupancy entropy and death concentration. Generator-level occupancy entropy has a positive rank association with preference score, while concentrated death locations have a negative association. The weakest pattern generator, `patternCount`, has median maximum progress of only 174.5 pixels and a max death-bin share of 0.63, meaning failures are highly concentrated early in the level. In contrast, `original` and `mariodpo` have median progress around 1126.5 and 988.0 pixels respectively, with lower death concentration.

The deployed analytics UI also supports level-level inspection beyond aggregate plots. Figure 4.8 shows the Detailed Level Statistics page, which combines the level view, player traces, death locations, and tag counts. This is important as a platform contribution: PCG Arena is not only a leaderboard, but also an instrument for diagnosing why a generator wins or loses.

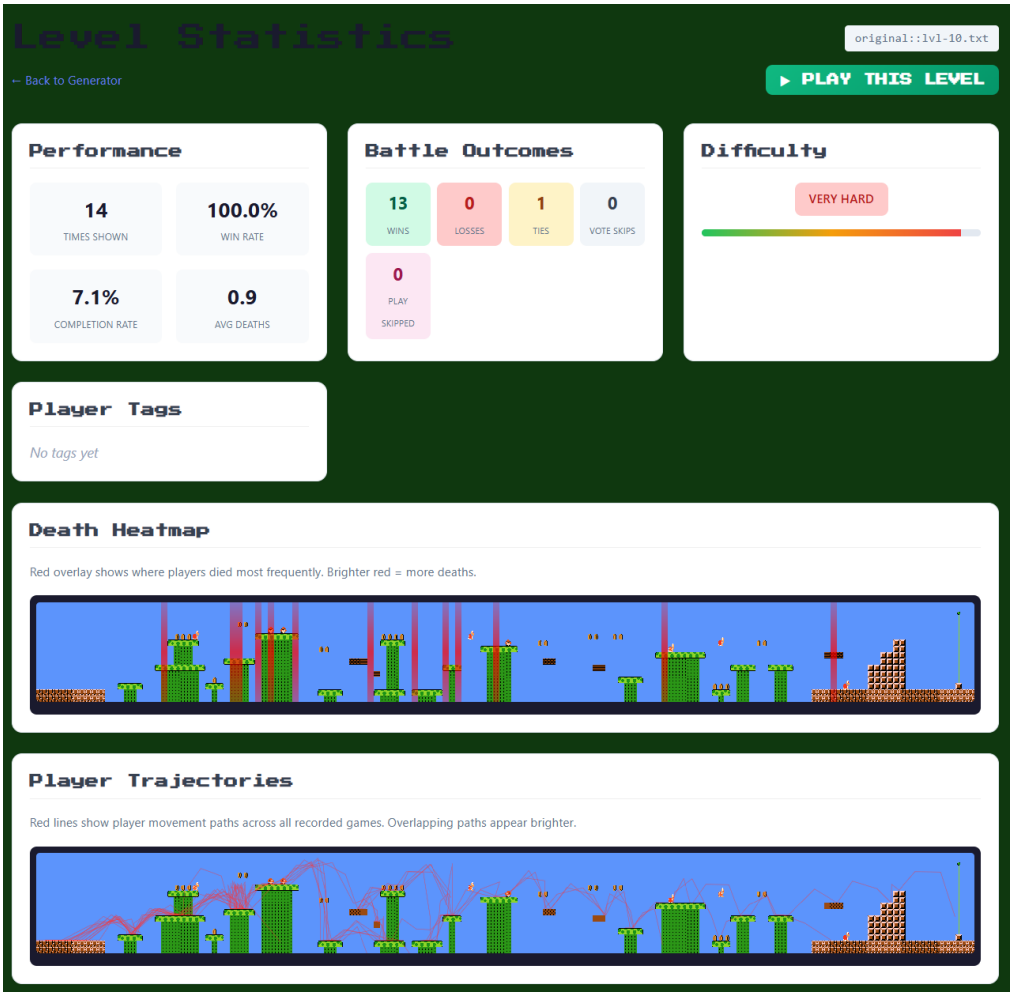


Figure 4.8: Detailed Level Statistics page from PCG Arena. The interface supports qualitative inspection of individual levels through trajectory overlays, death locations, tag counts, and level visualization.

4.5 RQ4: User Preference Heterogeneity

The anonymous-player data are sparse, and because players are identified only by browser/player IDs we know nothing about who they are—their age, background, or gaming experience are all unknown. The data are nonetheless useful for understanding engagement and possible preference heterogeneity. Figure 4.9 shows votes over time, votes per anonymous player ID, and cumulative vote contribution.

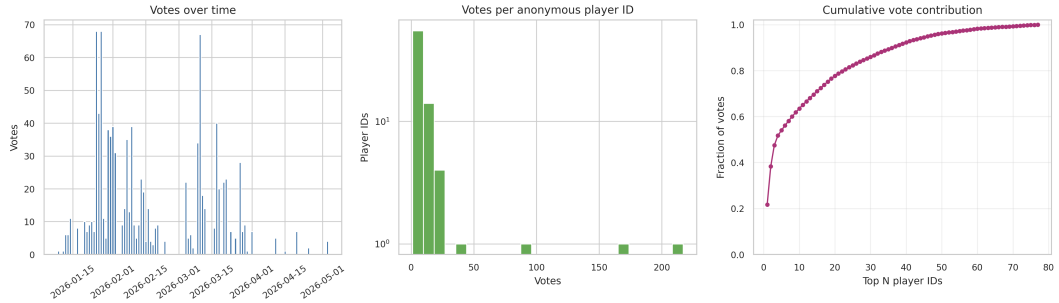


Figure 4.9: Engagement concentration in the anonymous player-ID data. The analysis uses browser/player IDs, so these counts should not be interpreted as verified unique humans.

The vote distribution is highly concentrated. Among the 77 anonymous player IDs appearing in the vote table, the median player ID contributes 6 votes, while the most active ID contributes 217 votes. The top 5 IDs account for 54% of the analyzed votes, and the top 10 account for 63.5%. This concentration is typical for voluntary web studies, but it matters for interpretation: aggregate generator scores reflect both broad participation and the preferences of a small number of highly engaged player IDs.

Figure 4.10 examines the player-by-generator preference matrix for anonymous player IDs with at least five votes. Each cell is the mean score assigned by a player ID when that generator appeared. Missing player-generator combinations are filled with the neutral value 0.5 only for visualization and clustering.

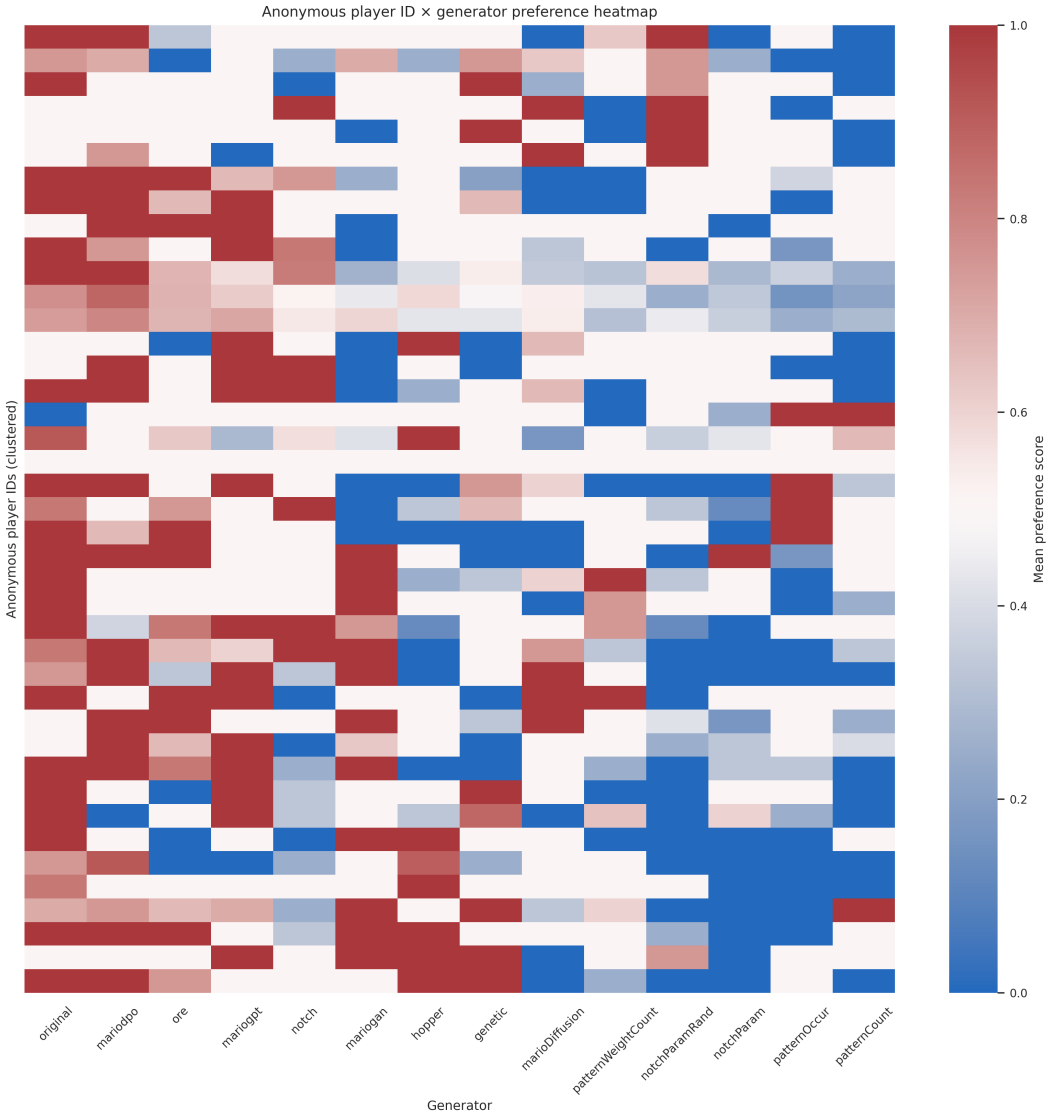


Figure 4.10: Anonymous player ID by generator preference heatmap for player IDs with at least five votes. Rows are grouped by hierarchical clustering - Ward's method on Euclidean distances between players' generator-score vectors. Columns follow the global generator ranking. The plot is exploratory because many player-generator combinations are sparse.

The heatmap suggests that, despite some variation, players largely agree on which generators are good and which are bad: most rows score the top-ranked generators highly and the bottom-ranked generators poorly, with only mild disagreement on the mid-table generators. This consistency across an anonymous and otherwise undescribed player population is itself a useful finding: it suggests that Super Mario Bros is a strong benchmark for PCG evaluation, because player preferences over levels are stable enough to produce a repeatable ranking rather than being dominated by individual taste.

4.6 RQ5: Tag Semantics and Failure Modes

Tags provide a lightweight qualitative layer over the binary vote. They are optional, sparse, and subjective, so they should not be treated as ground-truth labels. Their value is diagnostic: they help distinguish why a level wins or loses.

Figure 4.11 summarizes tag frequency, tag rate by generator, tag co-occurrence, and outcome distribution when each tag is present. The most common tags are `impossible` (87 assignments), `fun` (56), `too_hard` (55), `boring` (49), `creative` (39), `broken_graphics` (22), and `too_easy` (21).

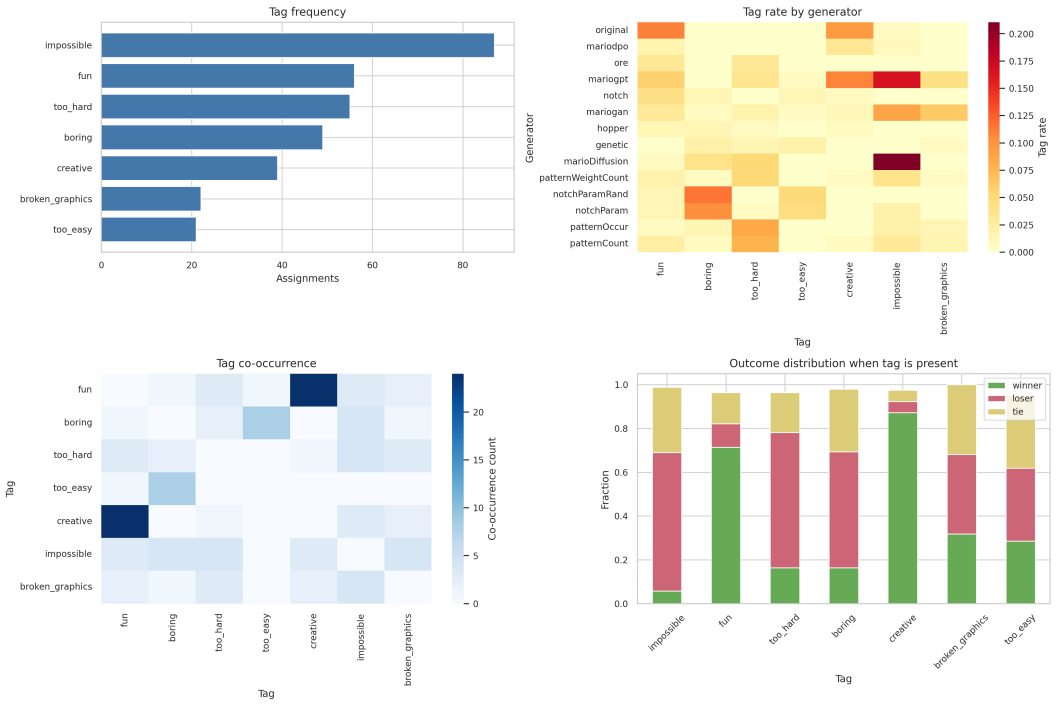


Figure 4.11: Tag semantics summary. The panels show tag frequency, tag rate by generator, tag co-occurrence, and the outcome distribution for tagged sides.

Positive tags are strongly aligned with preference outcomes. Sides tagged `creative` win 87% of the time and sides tagged `fun` win 71% of the time. Negative tags behave differently: `impossible`, `too_hard`, and `boring` are much more common on losing sides than winning sides. The tag-by-generator heatmap also makes generator failure modes interpretable: some generators lose because they are marked as impossible or too hard, while others are rejected as boring or visually broken.

4.7 Summary of Findings

PCG Arena provides a live human-preference measurement instrument for procedural level generators. It ranks generators through blind pairwise comparison, shows

stable head-to-head patterns, connects rankings to static expressive range, reveals behavioral differences through trajectories, exposes engagement concentration and possible preference diversity, and uses tags to explain qualitative failure modes.

The main empirical findings are:

1. **Preference ranking**: `original` and `mariodpo` are the top generators in the analyzed export, with nearly identical preference scores around 0.83.
2. **Generator families**: Pattern-based generators are consistently weak, while neural and constructive generators occupy the middle tiers.
3. **Static structure**: Generators occupy visibly distinct expressive ranges. Reward density, unique-column ratio, and distance to the original corpus are more informative than any single difficulty proxy.
4. **Gameplay behavior**: Trajectories reveal navigability and failure modes. Stronger generators tend to support broader movement footprints and less concentrated failure locations.
5. **Users**: Player votes are largely consistent across the anonymous population: the same generators tend to score high or low across players, suggesting that Super Mario Bros is a good benchmark whose preference signal is not dominated by individual taste.
6. **Tags**: Optional tags add semantic explanations to votes, distinguishing positive qualities such as `fun` and `creative` from failure modes such as `impossible`, `too_hard`, and `boring`.

These findings directly inform the design of the preference-aligned generation pipeline described in the following chapter.

Chapter 5

MarioDPO — Preference-Aligned Level Generation

5.1 Motivation and Approach

The previous chapter established two facts that motivate the final technical contribution of this thesis. First, blind pairwise votes collected through PCG Arena produce a stable ranking of Mario level generators: the original Nintendo levels remain the strongest baseline, but several modern generators approach their quality. Second, the raw telemetry and tag data reveal measurable correlates of preference—engagement duration, completion, vertical movement, style similarity, and the avoidance of frustrating hazards. MarioDPO uses these signals not only to *evaluate* generators, but also to *improve* one.

The central idea is to close the loop between human evaluation and procedural generation. Rather than optimising a generator for playability alone, or for an ad hoc diversity metric, MarioDPO fine-tunes a level generator on pairwise preferences. In this setting a generated level is treated as a sequence, a vote in PCG Arena provides a preferred and rejected completion, and Direct Preference Optimisation (DPO; Section 2.6.2) updates the generator so that preferred levels become more likely than rejected ones. This is the same alignment principle used in modern language-model training, applied to game content instead of natural language.

MarioDPO consists of four components:

1. a column-major level representation that preserves the vertical structure of Mario tilemaps while remaining compatible with autoregressive sequence modelling;
2. a base generator trained to model the syntax of Nintendo-style levels;
3. a neuro-symbolic *judge function* that converts the EDA findings into a scalable preference proxy; and

4. a DPO fine-tuning stage that mixes real PCG Arena votes with judge-labelled synthetic pairs.

The design deliberately separates **validity** from **preference**. Validity is enforced through representation choices, start/end constraints, post-processing, and optional A* filtering. Preference is learned from human votes and from the judge function. This separation is important: a level can be playable yet boring, or structurally diverse yet unpleasant. MarioDPO targets the stronger objective: generating levels that players are likely to prefer in blind A/B tests.

Figure 5.1 summarises the full pipeline. The original SMB levels provide the grammar prior, PCG Arena provides sparse but high-value human comparisons, the judge function expands these comparisons into a larger preference dataset, and DPO aligns the generator with the resulting preference signal.

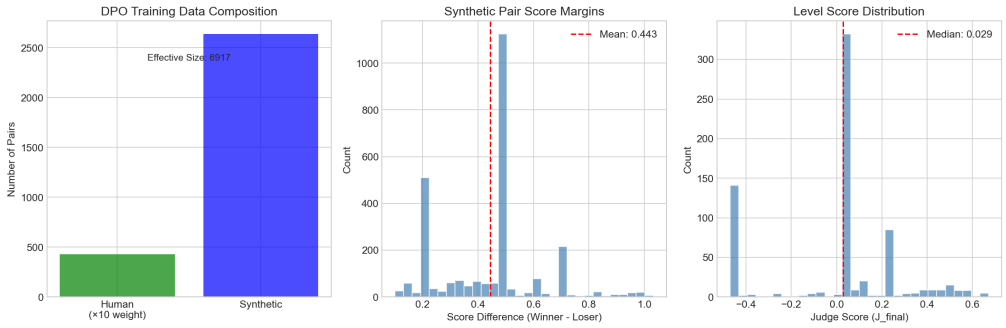


Figure 5.1: MarioDPO training data pipeline. Human PCG Arena votes are retained as high-weight preference pairs, while the judge function supplies additional synthetic pairs with lower weight. The combined dataset is used for DPO fine-tuning.

5.2 Column-Major Tokenisation

Mario levels in this thesis are represented as ASCII tilemaps: each level is a matrix of characters, with rows corresponding to vertical tile positions and columns corresponding to horizontal progress through the level. The standard height is 16 tiles; widths vary between generators, with the corpus average equal to 193.8 tiles. Across all seed levels the vocabulary contains 37 distinct tile characters, dominated by empty space (`-`), solid blocks (`X`), pipes (`|`, `<`, `>`, `[`, `]`), platforms (`%`), question blocks (`?`, `Q`), coins (`o`), and enemies such as Goombas (`g`) and Koopas (`k`).

A naive row-major serialisation would place horizontally adjacent tiles next to one another in the sequence, but would separate vertically adjacent tiles by an entire row length. This is undesirable for Mario levels, where local vertical relationships are essential: enemies stand on platforms, pipes occupy multiple rows, and jumpable structures depend on the configuration of several tiles in a column. MarioDPO therefore uses a column-major representation. Conceptually, the level is decomposed

into a sequence of 16-character columns,

$$c_t = (x_{1,t}, x_{2,t}, \ldots, x_{16,t}), \tag{5.1}$$

where $x_{r,t}$ is the tile at row r and column t. The autoregressive model then predicts the next column or the next tile in a flattened column-major sequence. This preserves vertical locality while still allowing standard left-to-right generation.

Table 5.1 summarises the representation choices. The same representation is also used by the implementation's column-level generator: n-gram contexts are sequences of full columns, and post-processing operates on columns when enforcing ground continuity, start safety, and flag placement.

Table 5.1: MarioDPO level representation and tokenisation choices.

Property	**Choice**
Level format	ASCII tilemap
Height	16 rows
Average width in corpus	193.8 tiles
Vocabulary size	37 tile characters
Sequence order	Column-major, left to right
Base context length	2048 tokens ($\approx$ 128 columns)
Style conditioning	Prefix token, e.g. `[STYLE:NINTENDO]`
Validity constraints	Safe start, bounded gaps, end flag, post-processing

The context length of 2048 tokens is sufficient to model long local segments of a Mario level. For levels exceeding this length, training examples are obtained by sliding a window over the column-major sequence. At inference time, generated segments are decoded back into 16-row tilemaps and then passed through the validity filters described in Section 5.4.3.

5.3 Judge Function Development

Human votes are the most reliable supervision signal in PCG Arena, but they are too sparse to train a sequence model directly. The January 2026 snapshot used in the MarioDPO experiments contains 473 usable non-tie votes. To expand this data without abandoning the human preference objective, we construct a judge function J that predicts the human preference quality of a level from telemetry and structural features. The judge is not intended to replace the final human evaluation; instead, it serves as a scalable preference proxy for generating additional DPO pairs.

The judge function is neuro-symbolic in the sense that it combines manually interpretable terms—style distance, gap penalty, verticality, and flow—with weights derived from empirical correlations in the PCG Arena data. Its design is guided by

three findings from Chapter 4: players prefer levels that hold attention longer, tags such as *fun* and *creative* correlate with win rate, and high-performing generators tend to resemble original Nintendo levels without simply copying them.

5.3.1 Verticality Reward (Experiment A)

The first component tests whether players prefer levels that encourage movement through the vertical space of the screen, rather than only horizontal running on flat ground. For each trajectory we compute the standard deviation of Mario's vertical coordinate,

$$\sigma_y = \sqrt{\frac{1}{T}\sum_{t=1}^{T}(y_t - \bar{y})^2}, \tag{5.2}$$

where T is the number of recorded trajectory samples. High σ_y indicates that the player moved across multiple elevations, typically because the level contains hills, platforms, pipes, or jumping sequences. We also compute a simple path-entropy proxy: the number of distinct tile coordinates visited by the player.

Experiment A found that verticality is positively associated with human preference. Across levels with sufficient data, σ_y correlates with win rate at $r_s = 0.257$ ($p = 0.019$), while path entropy correlates at $r_s = 0.275$. The effect is modest, but significant: players do not merely prefer easy levels; they prefer easy levels with some spatial variety. Figure 5.2 visualises this relationship.

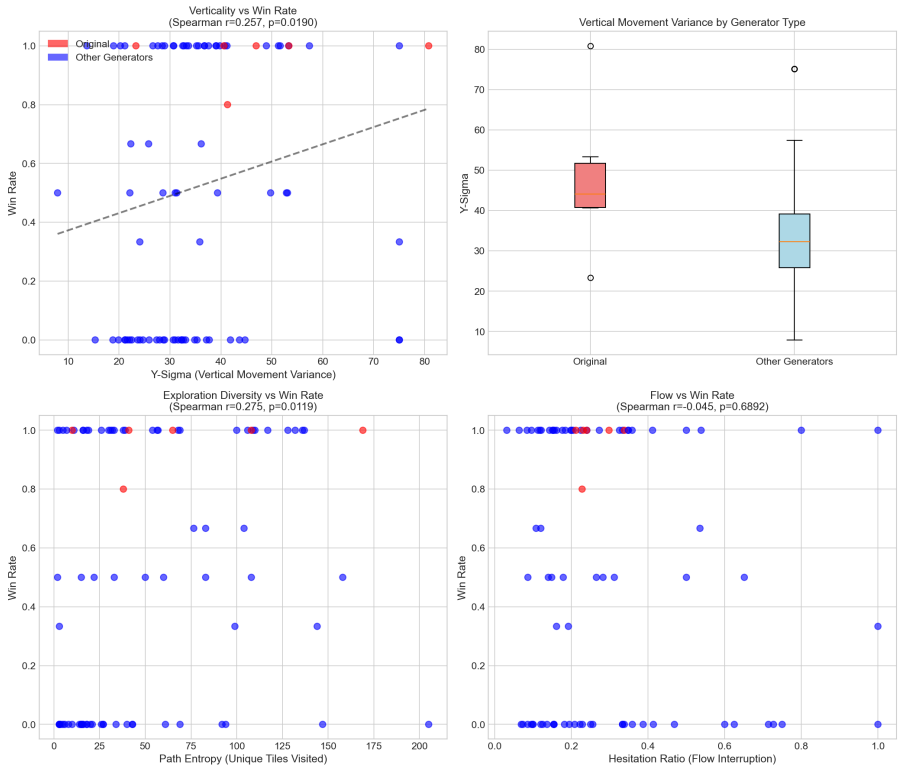


Figure 5.2: Experiment A: trajectory verticality and path entropy are positively associated with human win rate. MarioDPO uses σ_y as a reward term in the judge function.

This finding motivates the verticality term in J_{final}. Flat, safe levels may be playable, but they often lack the variety that players tag as *creative* or *fun*. Conversely, verticality is rewarded only after basic validity and flow constraints are considered, because vertical movement caused by repeated falls or deaths should not be interpreted as desirable exploration.

5.3.2 Hazard Hierarchy (Experiment B)

The second component addresses a limitation of aggregate difficulty measures. Chapter 4 showed that higher death counts are generally associated with lower preference, but not all sources of difficulty are equal. In Mario, enemies can be avoided, jumped on, or used as part of a satisfying action sequence. Gaps, by contrast, frequently create abrupt failures, especially in short blind evaluations where players have little time to learn a level.

The database snapshot used for the judge experiments did not contain reliable gap-density and enemy-density fields for all levels, so Experiment B used generator identity as a proxy. The three pattern-based generators produce many levels with fragmented ground and repeated gap structures. They achieved an average win rate of only 25.7%, compared with 54.2% for the remaining generators. This large gap

supports a hierarchy of hazards: pit-heavy level structure is penalised more strongly than moderate enemy placement.

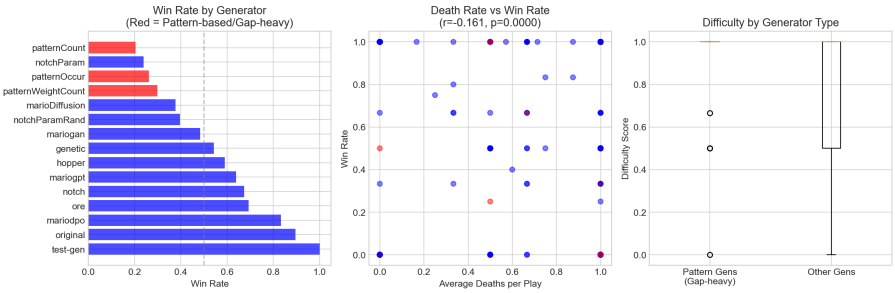


Figure 5.3: Experiment B: pattern-based generators form a low-performing cluster, supporting a strong gap/frustration penalty in the judge function.

MarioDPO encodes this result through a gap penalty. When explicit structural features are available, the penalty is proportional to gap density; when they are missing, pattern-generator membership is used as a conservative proxy. The judge also includes a flow term based on hesitation, defined as the fraction of trajectory intervals with near-zero horizontal velocity. Hesitation is weak by itself, but it captures a common failure mode of generated levels: the player is not dead, yet the level layout produces stopping, confusion, or repeated small corrections.

Experiment C attempted to measure whether deaths clustered at a small number of locations are more damaging than deaths spread throughout a level. The available data were insufficient for a stable death-entropy estimate, and the early-death rate correlation was weak ($r_s = -0.134$). Death entropy is therefore treated as a diagnostic feature rather than a primary term in the final judge used for DPO-pair construction.

5.3.3 Style Matching (Experiment D)

The third component captures the "Nintendo factor" observed throughout the arena results: human-authored original levels consistently outperform algorithmic generators. Rather than hard-coding individual tile ratios, MarioDPO estimates a style centroid from original levels in a telemetry-derived feature space. Let $\mathbf{z}$ be the feature vector of a level and let $\boldsymbol{\mu}_O$ and Σ_O denote the mean and covariance of the original-level feature vectors. The Mahalanobis distance to the original style is

$$D_M(\mathbf{z}) = \sqrt{(\mathbf{z} - \boldsymbol{\mu}_O)^\top \Sigma_O^{-1} (\mathbf{z} - \boldsymbol{\mu}_O)}. \tag{5.3}$$

Mahalanobis distance is preferable to Euclidean distance here because the features have different scales and correlations. A generator is not rewarded for matching one marginal statistic independently; it is rewarded for occupying the same joint region of feature space as original levels. The style reward is then defined as a bounded

inverse distance,

$$R_{style} = \frac{1}{1 + D_M}. \tag{5.4}$$

Experiment D confirms that this term is meaningful. Mahalanobis distance from the original centroid correlates negatively with win rate ($r_s = -0.279$, $p = 0.0105$), while the inverse style reward correlates positively at the same magnitude. Figure 5.4 shows the relationship.

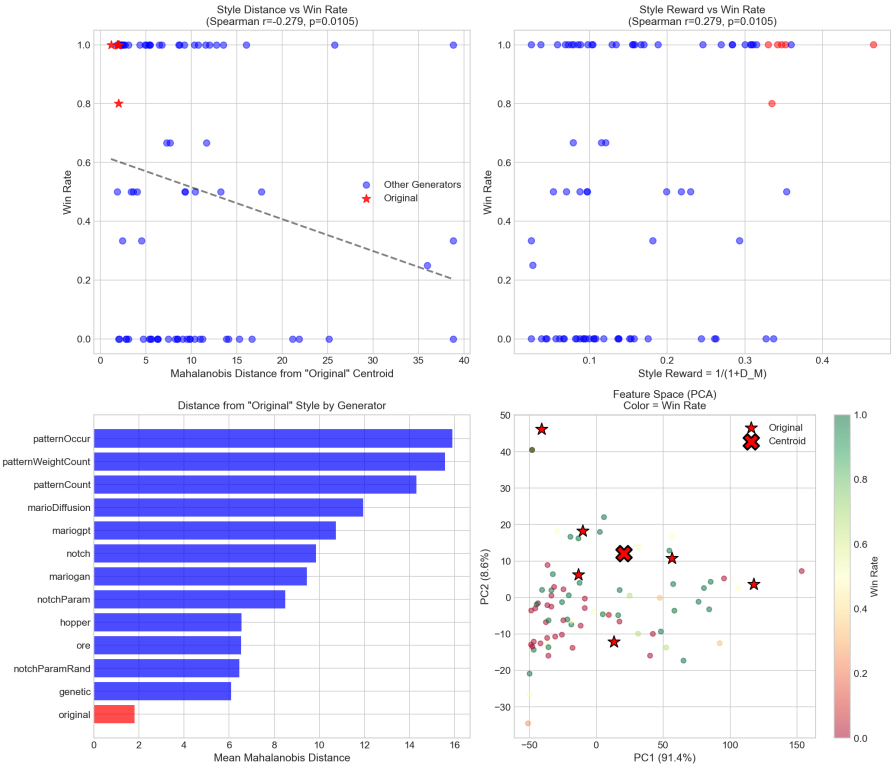


Figure 5.4: Experiment D: levels closer to the original-level centroid in feature space achieve higher win rates. MarioDPO uses this relationship as a style reward rather than as a hard constraint.

Importantly, the style term is not a copying objective. Original levels define a region of desirable structure, not a set of templates to reproduce. The DPO stage can still increase the probability of novel levels, provided that their telemetry and structural features remain close to the high-performing region.

5.3.4 Final Judge Function

The final judge function combines the above components in two stages. The first stage can be computed before expensive simulation, using style and hazard features:

$$J_{static} = w_{style}\frac{1}{1 + D_M} - w_{gap}\, GapPenalty + w_{comp}\, CompletionRate. \tag{5.5}$$

The second stage adds dynamic telemetry extracted from an A* or player trajectory:

$$J_{final} = J_{static} + w_{vert}\frac{\sigma_y}{50} + w_{flow}(1 - Hesitation). \tag{5.6}$$

The implemented weights are shown in Table 5.2. They were chosen from the relative strength and reliability of the experiments above: style and verticality receive moderate positive weights, flow receives a smaller positive weight, and gap-heavy structure receives the strongest penalty.

Table 5.2: Weights used by the MarioDPO judge function.

Term	**Weight**	**Rationale**
w_{style}	0.32	Original-centroid similarity, Experiment D
w_{vert}	0.26	Verticality reward, Experiment A
w_{flow}	0.10	Penalise hesitation and poor flow
w_{gap}	0.50	Strong penalty for gap-heavy/pattern-like failure modes
w_{comp}	0.20	Reward observed completion without making ease the sole target

To validate the combined judge, we aggregate J_{final} by generator and compare it with the corresponding human win rates. The Spearman correlation is $r_s = 0.736$ with $p = 0.0027$, a strong relationship given that the judge uses only telemetry and structural proxies rather than the votes themselves. Figure 5.5 shows the component distributions and the relationship between judge score and human preference.

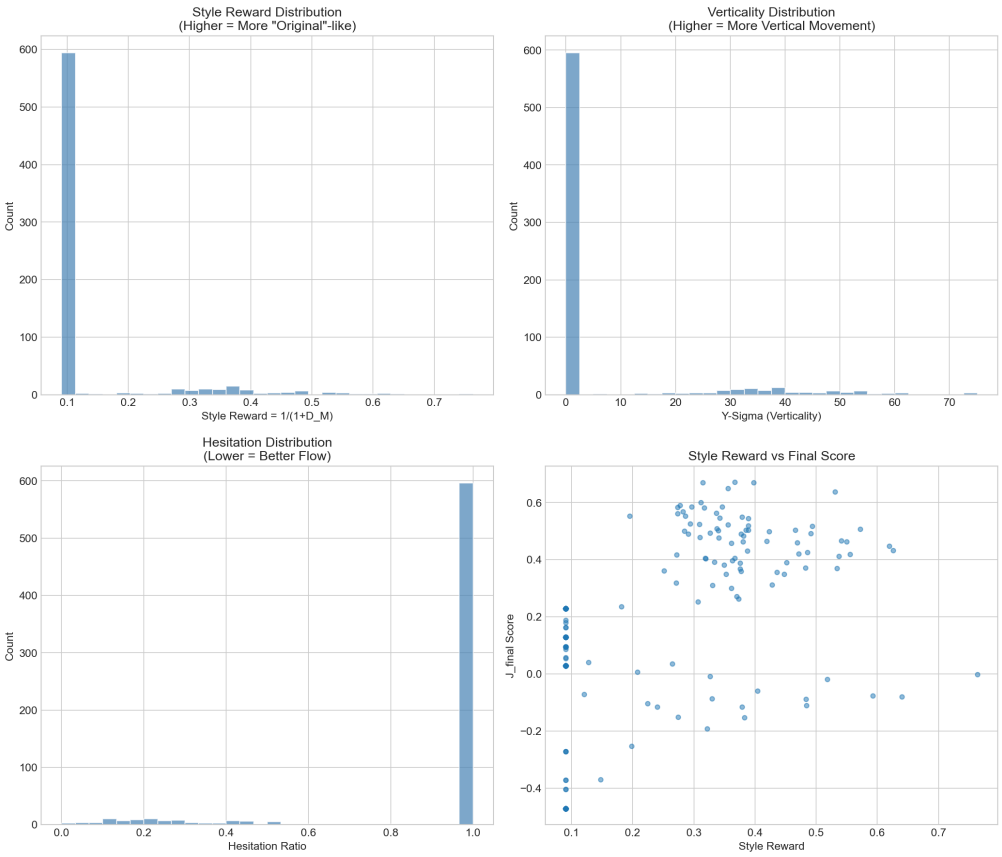


Figure 5.5: Judge-function component analysis. The final score combines style reward, trajectory verticality, hesitation, completion, and gap penalties. The resulting generator-level score correlates strongly with human win rate.

This validation justifies using J_{final} for Reinforcement Learning from AI Feedback (RLAIF): the judge can label many candidate level pairs cheaply, while real human votes remain the highest-weight examples in the DPO dataset.

5.4 Training Pipeline

5.4.1 Supervised Fine-Tuning (SFT)

The base policy π_{ref} is a GPT-2-style autoregressive decoder trained on column-major Mario sequences, following the language-modelling framing of MarioGPT [28]. The purpose of this stage is not yet to optimise preference; it is to learn the syntax of valid SMB levels: ground tends to occupy the lower rows, pipes have consistent multi-tile shapes, blocks float above reachable space, and enemies appear on supporting tiles.

The SFT corpus contains the 15 original Nintendo levels from the seed database, augmented by sliding windows over long levels. These levels are used as the refer-

ence style because they are the strongest human-authored baseline in PCG Arena and because Chapter 4 shows that the original generator remains the highest-rated system. Training uses a standard next-token objective:

$$\mathcal{L}_{SFT}(\theta) = -\sum_{t=1}^{T} \log \pi_\theta(x_t \mid x_{<t}, s), \tag{5.7}$$

where s is an optional style token such as `[STYLE:NINTENDO]`.

Using the original levels as the SFT corpus intentionally biases the model towards Nintendo-like structure. This is a conservative choice: PCG Arena's preference data is still small, so the reference policy should already produce plausible levels before DPO begins. DPO then moves this policy toward preference signals without allowing it to drift arbitrarily far from valid Mario syntax.

5.4.2 Preference Dataset Construction

The DPO dataset contains two sources of pairwise supervision.

Human pairs. Each non-tie PCG Arena vote is converted into a pair (y_w, y_l), where y_w is the level selected by the player and y_l is the rejected level. Votes marked as ties or skips are excluded because they do not define a strict preference. The January 2026 snapshot provides 473 such human pairs. Because these pairs are the only direct measurements of human preference, they are assigned a weight of 10 during training.

Synthetic pairs. To expand the dataset, candidate levels are scored with J_{final}. For a pair of levels (a, b), the higher-scoring level becomes the winner if the score margin exceeds 0.1. This margin avoids training on ambiguous pairs where the judge itself is nearly indifferent. The experiment pipeline produced 2,606 synthetic pairs with a mean score difference of 0.441. Synthetic examples receive unit weight.

The resulting dataset is summarised in Table 5.3. The total number of stored pairs is 3,079, but after applying the human-data weight the effective dataset size is 7,336 comparisons.

Table 5.3: Preference dataset used for MarioDPO fine-tuning.

Source	Pairs	Weight	Effective count
Human PCG Arena votes	473	10×	4,730
Judge-labelled synthetic pairs	2,606	1×	2,606
Total	**3,079**	—	**7,336**

The weighting scheme reflects the intended role of the judge. It increases data coverage and supplies clear negative examples, but it does not override human feedback. If the judge and a human vote disagree in similar regions of the level space, the human vote dominates the gradient through repeated sampling or example weighting.

5.4.3 DPO Training

The DPO objective is the one introduced in Equation 2.5. In the MarioDPO setting, the prompt x consists of the style token and any level prefix, while y_w and y_l are the preferred and rejected level continuations. The reference policy π_{ref} is the SFT model, and the trainable policy π_θ is initialised from it.

The training configuration used in the experiments is shown in Table 5.4. The value $\beta = 0.1$ controls how strongly DPO penalises divergence from the reference policy. A small learning rate is used because the dataset is preference-rich but small relative to typical language model corpora.

Table 5.4: MarioDPO fine-tuning configuration.

Parameter	**Value**
Base model	GPT-2-small-style decoder
Vocabulary	37 ASCII tile characters
Context length	2048 tokens
Batch size	32
DPO β	0.1
Learning rate	10^{-5}
Epochs	3
Human-pair weight	10×
Synthetic-pair weight	1×

Training increases the likelihood ratio of preferred levels relative to rejected levels, while the reference-policy term prevents catastrophic drift away from valid Mario syntax. Intuitively, if the SFT model assigns similar probability to two level continuations but humans or the judge prefer one, DPO shifts probability mass toward the preferred continuation. If the current model already strongly prefers the winner, the gradient is small.

Inference-time filtering. Preference alignment alone does not guarantee that every sampled level is valid. MarioDPO therefore uses rejection sampling at inference time. For each requested level the model generates a batch of $N = 10$ candidates. Candidates failing basic format checks are discarded; when simulation is available, an A* agent filters out unplayable candidates. The remaining candidates are scored

with J_{final}, and the highest-scoring candidate is selected. Finally, a deterministic post-processing pass ensures that Mario has a valid spawn point near the start, that a flag exists near the end, and that both regions contain solid ground. This "safety net" is deliberately separate from the learned model: the model learns preference, while the filter enforces hard validity constraints.

5.5 Implementation Details

The MarioDPO implementation is organised as a small experimental pipeline. The generator module implements column-wise generation and style conditioning. The core class learns n-gram transitions over complete 16-tile columns, while the style-conditioned variant exposes profiles such as *easy*, *medium*, *hard*, and *Nintendo*. These profiles adjust the probabilities of gaps, enemies, and platforms before the final validity pass. Although the DPO model is sequence-based, this column-level implementation is useful in two ways: it provides a strong syntax-preserving baseline and it supplies generated candidates for arena evaluation.

The experiment module implements the judge and preference-data construction. It loads exported PCG Arena statistics, extracts trajectory features (σ_y, path entropy, hesitation ratio, and maximum progress), fits the original-level centroid, scores all levels with J_{final}, and creates both human and synthetic preference pairs. The same module produces the diagnostic plots used in this chapter.

The post-processing module performs deterministic repairs that are better expressed as constraints than as learned behaviour. It removes duplicate Mario or flag tiles, places Mario within the first five columns, places the flag within the last five columns, and ensures solid ground at both boundaries. These checks make generated levels compatible with the PCG Arena engine and prevent trivial losses caused by missing spawn or termination markers.

5.6 Results

MarioDPO is evaluated in two complementary ways. First, the January 2026 MarioDPO experimental snapshot measures judge-function validity and compares the new generator with established baselines. Second, the May 2026 PCG Arena export analysed in Chapter 4 evaluates MarioDPO under the same blind pairwise-preference protocol as all other generators.

Controlled MarioDPO snapshot

Table 5.5 summarises the controlled snapshot. The original Nintendo levels remain the strongest baseline with an 89.6% win rate, but MarioDPO reaches 83.3%, out-

performing all other procedural generators in the comparison. MarioGPT reaches 63.8%, MarioGAN 48.4%, and the pattern-based generators average only 25.5%.

Table 5.5: Controlled MarioDPO experimental snapshot. Win rate is computed from PCG Arena pairwise outcomes in the January 2026 export.

Generator	**Win rate**	**Mean J_{final}**	**Style distance**
Original SMB1	89.6%	0.282	5.75
MarioDPO	83.3%	0.339	4.09
MarioGPT	63.8%	0.096	9.56
MarioGAN	48.4%	0.127	8.82
MarioDiffusion	37.8%	0.080	9.22
Pattern-based average	25.5%	−0.436	9.47

Two patterns are visible. First, MarioDPO has the lowest style distance among the procedural generators, indicating that the original-centroid reward successfully pulls it toward the human-authored region of feature space. Second, its mean judge score is higher than the original baseline even though its human win rate is lower. This is not a contradiction: the judge is a proxy trained from limited features, while human votes remain the final criterion. The discrepancy suggests that MarioDPO has learned many of the features captured by the judge, but still lacks some aspects of hand-authored Nintendo design.

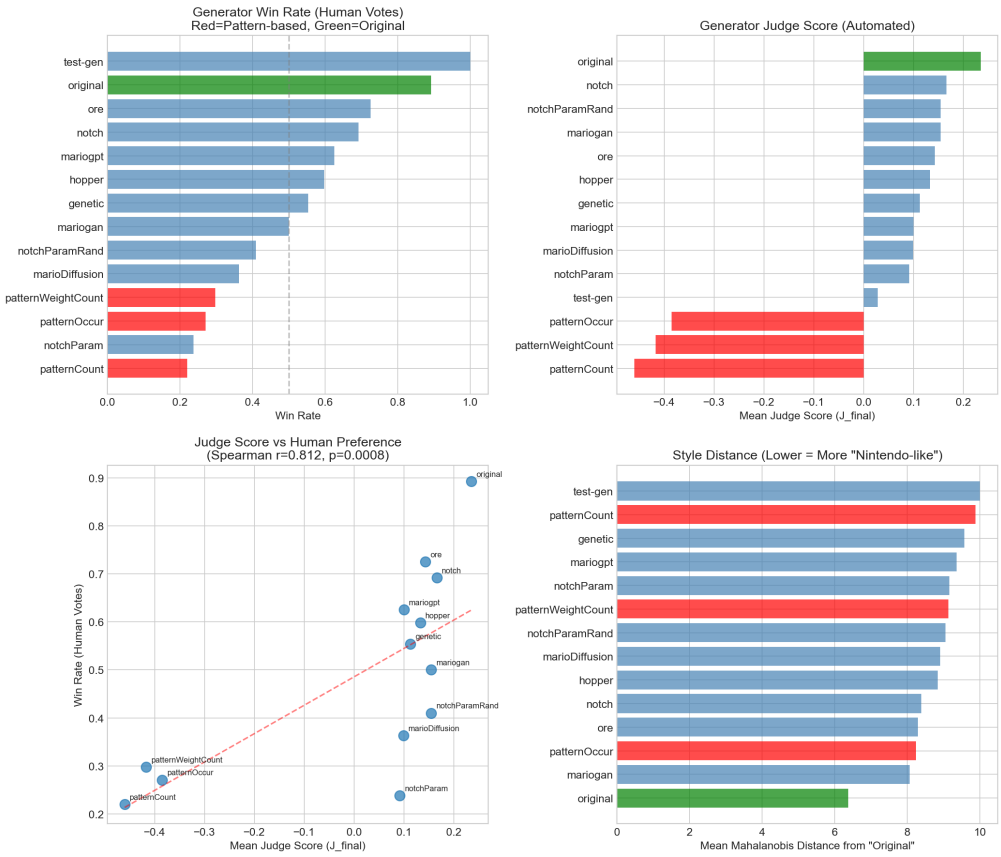


Figure 5.6: Generator-level analysis from the MarioDPO experiment snapshot. The judge score correlates strongly with human win rate, and MarioDPO forms the closest procedural competitor to the original levels.

Full arena snapshot

In the later May 2026 PCG Arena export, MarioDPO remains the strongest procedural generator. It obtains a preference score of 0.832, essentially tied with the original Nintendo levels at 0.833 and ahead of ORE (0.655) and MarioGPT (0.643). The pairwise matrix in Chapter 4 further shows that MarioDPO performs consistently across many direct matchups rather than benefiting only from weak opponents.

Figure 5.7: MarioDPO compared with other procedural generators. Across both the controlled experiment and the full arena snapshot, MarioDPO is the strongest PCG method and the closest competitor to the original human-authored levels.

The change from an 83.3% win rate in the controlled snapshot to a 0.832 preference score in the full arena export reflects the different aggregation methods: the later analysis counts ties as 0.5 and compares more opponents under broader player behaviour. The important qualitative conclusion is stable across both evaluations: preference alignment substantially improves generator quality, placing MarioDPO in the same top tier as the original SMB1 levels and above all other algorithmic generators evaluated in PCG Arena.

5.7 Analysis and Discussion

MarioDPO's performance can be explained by the interaction of three design choices.

First, the base model starts from a strong structural prior. Training on original levels and using a column-major representation gives the generator a bias toward valid, readable Mario geometry before preference optimisation begins. This avoids a common failure mode of pure search or random generation: the optimiser spends most of its capacity rediscovering basic syntax rather than improving quality.

Second, the judge function captures multiple dimensions of preference rather than a single difficulty metric. The final score rewards style similarity and vertical exploration, penalises gap-heavy frustration, and incorporates flow and completion. This is consistent with the EDA results: players prefer levels that are accessible, but accessibility alone is not enough. Levels must also sustain engagement and provide recognisable Mario-like structure.

Third, the preference dataset preserves the primacy of human judgement. Synthetic pairs make the dataset large enough for DPO, but the 10× weight on human

votes ensures that the model is ultimately anchored to PCG Arena preferences. This design is important because J_{final} is only an approximation. It correlates strongly with human win rate, but it cannot capture nostalgia, aesthetic surprise, or all forms of level readability.

Several limitations remain. The human preference dataset is still small by deep learning standards, and the synthetic pairs inherit any bias in the judge function. The trajectory features are derived from short play sessions and A* simulation, which may not reflect how experienced human players explore levels over longer periods. The style reward also privileges Nintendo-like levels; this is appropriate for the benchmark used in this thesis, but it may penalise innovative generators whose quality lies outside the original SMB1 style manifold. Finally, some potentially useful signals, such as death entropy and fine-grained tag usage, are limited by data sparsity.

Despite these limitations, MarioDPO demonstrates the main claim of this thesis: a persistent blind evaluation platform can do more than rank generators. It can produce the preference data needed to train better generators. PCG Arena supplies human comparisons, the judge function turns them into a scalable learning signal, and DPO converts that signal into improved procedural content. The result is a generator that approaches human-authored quality more closely than the other PCG methods tested in this work.

Chapter 6

Conclusions and Future Work

6.1 Summary of Contributions

tbd

6.2 Limitations

- **No evaluator quality model.** PCG Arena does not rate the quality of individual players' evaluations. Metrics such as time spent per level, decision latency, and tag usage frequency could serve as proxies for evaluation quality, but no such model is currently implemented.
- **Recognisability of original levels.** Players who are familiar with the original Nintendo levels may recognise them despite the blind setup, introducing a potential nostalgia or familiarity bias in favour of the human-authored baseline.

tbd

6.3 Future Directions

tbd

KOT(todo bibliography przejrzec na koniec)

Bibliography

[1] Ralph Allan Bradley and Milton E. Terry. Rank analysis of incomplete block designs: I. The method of paired comparisons. *Biometrika*, 39(3/4):324–345, 1952. doi: 10.1093/biomet/39.3-4.324.

[2] Wei-Lin Chiang, Lianmin Zheng, Ying Sheng, Anastasios Nikolas Angelopoulos, Tianle Li, Dacheng Li, Hao Zhang, Banghua Zhu, Michael Jordan, Joseph E. Gonzalez, and Ion Stoica. Chatbot arena: An open platform for evaluating LLMs by human preference. *arXiv preprint arXiv:2403.04132*, 2024. URL `https://arxiv.org/abs/2403.04132`.

[3] Paul F. Christiano, Jan Leike, Tom B. Brown, Miljan Martic, Shane Legg, and Dario Amodei. Deep reinforcement learning from human preferences. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017. URL `https://arxiv.org/abs/1706.03741`. arXiv:1706.03741.

[4] Steve Dahlskog and Julian Togelius. Patterns as objectives for level generation. In *Proceedings of the Second Workshop on Design Patterns in Games*, 2013. URL `http://julian.togelius.com/Dahlskog2013Patterns.pdf`.

[5] Steve Dahlskog and Julian Togelius. Procedural content generation using patterns as objectives. In *Applications of Evolutionary Computation (EvoApplications)*, volume 8602 of *Lecture Notes in Computer Science*, pages 325–336. Springer, 2014. doi: 10.1007/978-3-662-45523-4_27.

[6] Arpad E. Elo. *The Rating of Chessplayers, Past and Present*. Arco, 1978.

[7] Mark E. Glickman. Parameter estimation in large dynamic paired comparison experiments. *Journal of the Royal Statistical Society: Series C (Applied Statistics)*, 48(3):377–394, 1999. doi: 10.1111/1467-9876.00120.

[8] Mark E. Glickman. Example of the Glicko-2 system. Technical report, Boston University, 2012. URL `http://www.glicko.net/glicko/glicko2.pdf`.

[9] Britton Horn, Steve Dahlskog, Noor Shaker, Gillian Smith, and Julian Togelius. A comparative evaluation of procedural level generators in the Mario AI framework. In *Proceedings of the 9th International Conference on the Foundations of Digital Games (FDG)*, 2014. URL `https://www.fdg2014.org/papers/fdg2014_paper_14.pdf`.

[10] Sergey Karakovskiy and Julian Togelius. The Mario AI benchmark and competitions. *IEEE Transactions on Computational Intelligence and AI in Games*, 4 (1):55–67, 2012. doi: 10.1109/TCIAIG.2012.2188528.

[11] Ahmed Khalifa, Philip Bontrager, Sam Earle, and Julian Togelius. PCGRL: Procedural content generation via reinforcement learning. In *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment (AIIDE)*, 2020. URL `https://ojs.aaai.org/index.php/AIIDE/article/view/7416`. arXiv:2001.09212.

[12] José Rafael Hernández Mariño, Walber Moreira Penna Reis, and Levi H. S. Lelis. An empirical evaluation of evaluation metrics of procedurally generated Mario levels. In *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment (AIIDE)*, 2015. URL `https://ojs.aaai.org/index.php/AIIDE/article/view/12785`.

[13] Peter Mawhorter and Michael Mateas. Procedural level generation using occupancy-regulated extension. In *Proceedings of the IEEE Conference on Computational Intelligence and Games (CIG)*, pages 351–358. IEEE, 2010. doi: 10.1109/ITW.2010.5593333.

[14] Seong-Geon Nam, Chih-Hung Hsueh, Pasit Rerkjirattikal, and Kokolo Ikeda. Using RL to generate levels of Super Mario Bros with quality and diversity. *IEEE Transactions on Games*, 16(4):807–820, 2024.

[15] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul F. Christiano, Jan Leike, and Ryan Lowe. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. URL `https://arxiv.org/abs/2203.02155`. arXiv:2203.02155.

[16] Christopher Pedersen, Julian Togelius, and Georgios N. Yannakakis. Modeling player experience for content creation. *IEEE Transactions on Computational Intelligence and AI in Games*, 2(1):54–67, 2010.

[17] Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. URL `https://arxiv.org/abs/2305.18290`. arXiv:2305.18290.

[18] Anurag Sarkar and Seth Cooper. Level difficulty and player skill prediction in human computation games. In *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment (AIIDE)*, 2017. URL `https://ojs.aaai.org/index.php/AIIDE/article/view/12948`.

[19] Jacob Schrum, Olivia Kilday, Emilio Salas, Bess Hagan, and Reid Williams. Text-to-level diffusion models with various text encoders for Super Mario Bros. In *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment (AIIDE)*, pages 110–120, 2025. doi: 10.1609/aiide.v21i1.36815. URL `https://ojs.aaai.org/index.php/AIIDE/article/view/36815`.

[20] Noor Shaker, Georgios N. Yannakakis, and Julian Togelius. Towards automatic personalized content generation for platform games. In *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment (AIIDE)*, pages 63–68, 2010. doi: 10.1609/aiide.v6i1.12399.

[21] Noor Shaker, Julian Togelius, Georgios N. Yannakakis, Ben Weber, Tomoyuki Shimizu, Tomonori Hashiyama, Nathan Sorenson, Philippe Pasquier, Peter Mawhorter, Glen Takahashi, Gillian Smith, and Robin Baumgarten. The 2010 Mario AI championship: Level generation track. *IEEE Transactions on Computational Intelligence and AI in Games*, 3(4):332–347, 2011. doi: 10.1109/TCIAIG.2011.2166267.

[22] Noor Shaker, Georgios N. Yannakakis, and Julian Togelius. Feature analysis for modeling game content quality. In *Proceedings of the IEEE Conference on Computational Intelligence and Games (CIG)*, 2011. doi: 10.1109/CIG.2011.6031998.

[23] Noor Shaker, Miguel Nicolau, Georgios N. Yannakakis, Julian Togelius, and Michael O'Neill. Evolving levels for Super Mario Bros using grammatical evolution. In *Proceedings of the IEEE Conference on Computational Intelligence and Games (CIG)*, pages 304–311, 2012. doi: 10.1109/CIG.2012.6374170.

[24] Noor Shaker, Julian Togelius, and Mark J. Nelson. *Procedural Content Generation in Games: A Textbook and an Overview of Current Research*. Springer, 2016. doi: 10.1007/978-3-319-42716-4. URL `https://www.pcgbook.com`.

[25] Gillian Smith and Jim Whitehead. Analyzing the expressive range of a level generator. In *Proceedings of the 2010 Workshop on Procedural Content Generation in Games (PCGames)*, pages 4:1–4:7. ACM, 2010. doi: 10.1145/1814256.1814260.

[26] Rion Snow, Brendan O'Connor, Daniel Jurafsky, and Andrew Y. Ng. Cheap and fast — but is it good? Evaluating non-expert annotations for natural language tasks. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 254–263. Association for Computational Linguistics, 2008. URL `https://aclanthology.org/D08-1027/`.

[27] Nisan Stiennon, Long Ouyang, Jeffrey Wu, Daniel M. Ziegler, Ryan Lowe, Chelsea Voss, Alec Radford, Dario Amodei, and Paul F. Christiano. Learning

to summarize with human feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020. URL https://arxiv.org/abs/2009.01325. arXiv:2009.01325.

[28] Shyam Sudhakaran, Miguel González-Duque, Claire Glanois, Matthias Freiberger, Elias Najarro, and Sebastian Risi. MarioGPT: Open-ended text2level generation through large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. URL https://papers.neurips.cc/paper_files/paper/2023/file/a9bbeb2858dfbdbd4c19814e5d80ec60-Paper-Conference.pdf. arXiv:2302.05981.

[29] Adam Summerville and Michael Mateas. Super Mario as a string: Platformer level generation via LSTMs. In *Proceedings of the 1st International Joint Conference of DiGRA and FDG*, 2016. doi: 10.26503/dl.v2016i1.752. arXiv:1603.00930.

[30] Adam Summerville, José Rafael Hernández Mariño, Sam Snodgrass, Santiago Ontañón, and Levi H. S. Lelis. Understanding Mario: An evaluation of design metrics for platformers. In *Proceedings of the 12th International Conference on the Foundations of Digital Games (FDG)*, 2017. doi: 10.1145/3102071.3102080.

[31] Adam Summerville, Sam Snodgrass, Matthew Guzdial, Christoffer Holmgård, Amy K. Hoover, Aaron Isaksen, Andy Nealen, and Julian Togelius. Procedural content generation via machine learning (PCGML). *IEEE Transactions on Games*, 10(3):257–270, 2018. doi: 10.1109/TG.2018.2846639.

[32] Julian Togelius, Georgios N. Yannakakis, Kenneth O. Stanley, and Cameron Browne. Search-based procedural content generation: A taxonomy and survey. *IEEE Transactions on Computational Intelligence and AI in Games*, 3(3):172–186, 2011. doi: 10.1109/TCIAIG.2011.2148116.

[33] Vanessa Volz, Jacob Schrum, Jialin Liu, Simon M. Lucas, Adam Smith, and Sebastian Risi. Evolving Mario levels in the latent space of a deep convolutional generative adversarial network. In *Proceedings of the Genetic and Evolutionary Computation Conference (GECCO)*, pages 221–228. ACM, 2018. doi: 10.1145/3205455.3205517.

[34] Georgios N. Yannakakis and Julian Togelius. Experience-driven procedural content generation. *IEEE Transactions on Affective Computing*, 2(3):147–161, 2011. doi: 10.1109/T-AFFC.2011.6.

[35] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-Judge with

MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. URL `https://arxiv.org/abs/2306.05685`. arXiv:2306.05685.

[36] Daniel M. Ziegler, Nisan Stiennon, Jeffrey Wu, Tom B. Brown, Alec Radford, Dario Amodei, Paul Christiano, and Geoffrey Irving. Fine-tuning language models from human preferences. *arXiv preprint arXiv:1909.08593*, 2019. URL `https://arxiv.org/abs/1909.08593`.